

Análisis de single-cell RNA-seq

**Bioinformática y Estadística 3 · Licenciatura en Ciencias Genómicas, UNAM ·
Semestre 2027-1**

Dra. Yalbi Itzel Balderas-Martínez

14 de agosto de 2026

Tabla de contenidos

Presentación	6
Qué vas a poder hacer al terminar	6
Qué se da por sabido	7
Cómo está organizado el material	7
Herramientas	7
Licencia y cita	7
 I Antes de tocar los datos	 9
 1 Fundamentos de single-cell. Tecnologías y plataformas	 10
1.1 Cómo se desarrolla la sesión	10
1.2 Caso de aplicación	10
1.3 Desarrollo	11
1.3.1 La unidad de medición decide la pregunta	11
1.3.2 Bulk, célula única y espacial	11
1.3.3 El flujo completo y sus modos de falla	12
1.3.4 Tecnologías por principio de captura	12
1.3.5 Qué mide y qué no mide cada química	13
1.3.6 Células íntegras o núcleos aislados	13
1.3.7 Anatomía de una lectura de 10x	14
1.3.8 Cuántas células y a qué profundidad	14
1.4 Errores comunes	14
1.5 Recursos	15
1.6 Si algo falla	15
1.7 Referencias	16
 2 Diseño experimental	 17
2.1 Cómo se desarrolla la sesión	17
2.2 Caso de aplicación	17
2.3 Desarrollo	18
2.3.1 El experimento que no se pudo salvar	18
2.3.2 Réplica biológica, réplica técnica y la trampa de la célula	18
2.3.3 Qué se hace en su lugar	19
2.3.4 Confusión entre variables	19
2.3.5 Potencia: tres cosas que compiten por el mismo presupuesto	20
2.3.6 Multiplexado	21
2.4 Errores comunes	22
2.5 Recursos	22
2.6 Si algo falla	22

2.7	Referencias	23
II	Del FASTQ a un objeto en el que se puede confiar	24
3	De FASTQ a matriz	25
3.1	Cómo se desarrolla la sesión	25
3.2	Caso de aplicación	25
3.3	Desarrollo	26
3.3.1	De dónde sale un número de la matriz	26
3.3.2	Qué hace un cuantificador	26
3.3.3	Los archivos que llegan de la secuenciación	27
3.3.4	Trabajar en un clúster: enviar y esperar	28
3.3.5	Los comandos, de principio a fin	28
3.3.6	Cuatro cuantificadores y por qué no dan lo mismo	30
3.4	Errores comunes	30
3.5	Recursos	31
3.6	Si algo falla	31
3.7	Referencias	32
4	De matriz a objeto listo. Rutas de importación y estructuras de datos	33
4.1	Cómo se desarrolla la sesión	33
4.2	Caso de aplicación	33
4.3	Desarrollo	34
4.3.1	Dos matrices y una decisión	34
4.3.2	Por qué la matriz es dispersa	34
4.3.3	Anatomía del objeto	39
4.3.4	Las siete rutas de importación	41
4.3.5	Las cuatro preguntas obligatorias	42
4.3.6	Llamado de células	42
4.3.7	RNA ambiental	45
4.4	Errores comunes	45
4.5	Recursos	46
4.6	Si algo falla	47
4.7	Referencias	47
5	Control de calidad: del experimento a los datos	48
5.1	Cómo se desarrolla la sesión	48
5.2	Caso de aplicación	48
5.3	Desarrollo	49
5.3.1	El tipo celular que era el protocolo	49
5.3.2	La cadena, decisión por decisión	49
5.3.3	Las métricas por célula, leídas como evidencia	50
5.3.4	Umbrales: de dónde tienen que salir	50
5.3.5	Dobletes	51
5.3.6	El control de calidad no se hace una sola vez	51
5.4	Errores comunes	51
5.5	Recursos	52

5.6	Si algo falla	52
5.7	Referencias	53

III Autonomía y cierre 54

6 Práctica integradora y arranque del proyecto 55

6.1	Cómo se desarrolla la sesión	55
6.2	Caso de aplicación	55
6.3	Desarrollo	56
6.3.1	Por qué esta sesión existe	56
6.3.2	Qué se versiona y qué no	56
6.3.3	La ruta absoluta es el primer fallo	56
6.3.4	Semillas y registro de sesión	57
6.3.5	Las cuatro preguntas	57
6.3.6	Para quién se escribe un análisis reproducible	58
6.3.7	La tabla de decisiones	58
6.3.8	Verificación cruzada	58
6.4	La conclusión de la sesión	59
6.5	Requisito de salida	59
6.6	Calendario del proyecto	59
6.7	Errores comunes	59
6.8	Recursos	60
6.9	Si algo falla	60
6.10	Referencias	61

IV Anexos 62

Anexo: entornos de trabajo 63

Antes de empezar	63
Paso 1 · Entrar al servidor	63
Paso 2 · Cargar el entorno del curso	63
Paso 3 · Comprobar que todo está	64
Paso 4 · Tu espacio de trabajo	64
Paso 5 · La regla más importante	65
Paso 6 · Mandar un trabajo a la cola	65
Comandos que vas a usar todo el semestre	66
Si algo falla	66
Cuando pidas ayuda	67
Si prefieres tu propia computadora	67
Qué se puede montar y qué no	67
Paso 1 · Instalar R	68
Paso 2 · Los paquetes de R	68
Paso 3 · Instalar Python con conda	69
Tres trampas conocidas	70
Si formas parte del equipo docente	70

Anexo: glosario 71

Anexo: rutas de importación y estructuras de datos	72
Tabla maestra	72
Los tres objetos	72
Las trampas, por origen	73
Cell Ranger: raw frente a filtered	73
Objetos de práctica: ya vienen limpios	73
GEO: lo que el autor haya querido subir	74
CELLxGENE: estandarizado, pero X no son cuentas	74
El .rds de un colaborador	74
Incompatibilidad entre versiones, en general	74
Cómo detectarlo, de lo más rápido a lo más lento	75
Y la parte que sí está en tus manos	75
Las cuatro preguntas obligatorias	75
Anexo: ¿réplica biológica o técnica?	76
Antes de buscar tu caso en la lista	76
Humanos · donantes y muestreo	77
Gemelos	77
Animales de laboratorio	78
Cultivo celular y líneas	79
Organoides	80
Microbiología	81
Plantas	82
Muestreo ambiental y de campo	82
La capa técnica de la secuenciación	82
Específico de single-cell	83
Los cinco errores que más aparecen	84
Cuando tu caso no está en la lista	84
Referencias	85

Presentación



LICENCIATURA EN CIENCIAS GENÓMICAS

ENES Juriquilla, UNAM · Laboratorio de Biología Computacional, INER

Este manual acompaña al **módulo de *single-cell* RNA-seq** de la materia *Bioinformática y Estadística 3* de la Licenciatura en Ciencias Genómicas de la ENES Juriquilla, UNAM, semestre 2027-1, impartido en el Laboratorio de Biología Computacional (LABBIC) del Instituto Nacional de Enfermedades Respiratorias (INER).

Son ocho sesiones, del 18 de agosto al 17 de septiembre de 2026.

Qué vas a poder hacer al terminar

- Decidir si tu pregunta necesita resolución de célula única, y qué plataforma la responde.
- Diagnosticar un diseño experimental antes de secuenciar, no después.
- Moverte entre `SingleCellExperiment`, Seurat y AnnData sin perder metadata.
- Construir una matriz de conteos y justificar qué gotas contienen célula.
- Fijar umbrales de control de calidad a partir de tus datos y no de un tutorial.
- Reconocer un efecto de lote y saber cuándo corregirlo borra tu resultado.

Qué se da por sabido

La materia tiene prerequisites: Linux básico, programación básica en R, fundamentos de estadística, uso básico de GitHub y experiencia previa en análisis de transcriptoma. Este módulo se apoya en todos ellos desde la primera sesión.

Cómo está organizado el material

Recurso	Dónde	Para qué
Diapositivas	<code>slides/</code>	lo que se proyecta en clase
Este manual	<code>manual/</code>	la versión escrita, para estudiar
Notebooks	<code>notebooks/R/</code> y <code>notebooks/python/</code>	lo que corres tú
Proyecto final	<code>tareas/proyecto-final/</code>	la evaluación del módulo

Herramientas

El módulo es deliberadamente bilingüe en herramientas. No son tres alternativas entre las que elegir, son tres dialectos que vas a encontrar en la literatura y en los datos que te manden:

- **Bioconductor** / **OSCA** es el eje. Es el que hace explícita la estadística, y la materia se llama *Bioinformática y Estadística 3*.
- **Seurat v5** aparece en todas las prácticas, porque es lo que se usa en los módulos que siguen a este.
- **Scanpy** / **AnnData** se usa acotado: interoperabilidad y detección de dobles. El `.h5ad` es el formato en que te van a llegar los datos.

El razonamiento completo, con lo que se decidió dejar fuera, está en la capa de conocimiento del repositorio (`okf/`).

Licencia y cita

Todo el material de este repositorio —texto, figuras, código y datos propios— está bajo **CC BY-NC 4.0**: se puede usar, adaptar y traducir libremente, **con crédito y sin fines comerciales**.

El crédito va a quienes escribieron el material que se reutiliza, no solo a la titular del módulo: la autoría de cada archivo consta en el historial de git del repositorio. La cita del conjunto está en `CITATION.cff`, y el texto completo de la licencia con sus exclusiones, en `LICENSE`.

i Asistencia de inteligencia artificial

Parte de este material se produjo con asistencia de **Claude (Anthropic)**, modelo **Opus 5**, bajo dirección y revisión de la autora: redacción y depuración del código de los notebooks, borradores de texto y las comprobaciones automáticas del repositorio. El diseño docente y la

responsabilidad sobre el contenido son de la autora. El detalle está en el **README** del repositorio.

Parte I

Antes de tocar los datos

1 Fundamentos de single-cell. Tecnologías y plataformas

Sesión 01 · Martes 18 de agosto de 2026 · sesión teórica, 120 minutos Imparten: Yalbi Itzel Balderas Martínez (30 min) y Luis Alberto Meza Cova (90 min)

i Objetivo

Distinguir que preguntas biológicas responde el análisis de célula única y cuáles no; ubicar las etapas del flujo completo de análisis y sus modos de falla; y comparar las principales tecnologías de secuenciación de célula única para elegir de forma justificada la adecuada según la pregunta biológica, el tipo de muestra y el presupuesto.

1.1. Cómo se desarrolla la sesión

Clase teórica con discusión dirigida, en dos bloques. Primero, panorama comparado de transcriptómica *bulk*, de célula única y espacial: qué resuelve cada una y a qué costo; y recorrido del flujo completo de la muestra a la conclusión biológica, señalando en cada etapa su modo de falla típico. Después, tecnologías por principio de captura: gota (10x Genomics 3' y 5'), placa (Smart-seq2 y Smart-seq3) e indexado combinatorial; células íntegras frente a núcleos aislados; qué mide y qué no mide cada química; y anatomía de una lectura de 10x (código de barras de célula, identificador molecular único e inserto). Actividad: en equipos, elegir y justificar la plataforma adecuada para tres escenarios experimentales contrastantes, completando una tabla comparativa.

- **6 bloques** de trabajo en 120 minutos, con descansos entre ellos
- Se alterna explicación y trabajo propio

1.2. Caso de aplicación

Caso de aplicación: el promedio que escondía dos poblaciones. Es la situación más frecuente en la que un laboratorio decide pasar de *bulk* a célula única. Un experimento de RNA-seq *bulk* reporta una bajada de expresión tras un tratamiento. Al repetirlo en célula única se descubre que ninguna célula bajo su expresión: lo que cambió fue la abundancia relativa de dos poblaciones. La conclusión original no era falsa en los datos, era falsa en la interpretación. Sirve para introducir la noción de que la unidad de medición determina la pregunta que se puede contestar, y regresa el 20 de agosto cuando se discuta la unidad de análisis.

1.3. Desarrollo

1.3.1. La unidad de medición decide la pregunta

El caso que abre la sesión no es una anécdota: es la razón por la que existe la técnica. Conviene verlo con números.

Un tejido con dos poblaciones, A y B. El gen X se expresa alto en A y bajo en B. Ninguna célula cambia su expresión tras el tratamiento; lo único que cambia es la proporción.

	Antes	Después
Población A, con X alto	70 %	30 %
Población B, con X bajo	30 %	70 %
Promedio medido en <i>bulk</i>	alto	bajo

El experimento *bulk* reporta una caída de X y el reporte es **numéricamente correcto**. Lo que es falso es la conclusión que se le cuelga: «el tratamiento reprime X». El promedio de un tejido está pesado por la abundancia de sus poblaciones, y un cambio de composición se lee igual que un cambio de regulación.

De aquí sale la idea que sostiene el módulo entero: **la unidad de medición determina qué preguntas se pueden contestar**. En *bulk*, la unidad es la muestra. En célula única, se mide una distribución entre células. Ningún reanálisis, por sofisticado que sea, recupera una distinción que el diseño no midió.

1.3.2. Bulk, célula única y espacial

Las tres miden expresión y las tres se justifican en contextos distintos.

	<i>Bulk</i>	Célula única	Espacial
Unidad de medición	la muestra	la célula o el núcleo	el punto o la célula, con coordenadas
Qué resuelve	expresión promedio del tejido	heterogeneidad y composición	organización en el tejido
Qué pierde	quién expresa qué	dónde estaba cada célula	resolución celular o profundidad, según la plataforma
Costo relativo	bajo	alto	alto

La consecuencia práctica es que **célula única no sustituye al *bulk***. Si la pregunta es si X cambia entre dos condiciones y no importa en qué tipo celular, el RNA-seq *bulk* con réplicas biológicas suficientes es más barato y estadísticamente más potente. Célula única se justifica cuando importa la composición, la heterogeneidad dentro de una población, o la identificación de tipos y estados celulares.

⚠ Una sola muestra con 10 000 células sigue siendo $n = 1$

Las células de un mismo donante no son réplicas biológicas independientes. La potencia para comparar condiciones la dan las **muestras**, no las células. Este es el tema central del Capítulo 2 y el error más caro del campo.

1.3.3. El flujo completo y sus modos de falla

Cada etapa del flujo tiene una forma característica de fallar, y casi todas se ven —o se dejan de ver— mucho después de haber ocurrido.

Etapas	Modo de falla típico	Se aborda en
Toma de muestra y disociación	firma de estrés por disociación; sesgo de composición	Capítulo 5
Captura y secuenciación	dobletes; RNA ambiental; profundidad insuficiente	Capítulo 5
De FASTQ a matriz	referencia o química mal declaradas	Capítulo 3
Llamado de células	gotas vacías contadas como células	Capítulo 4
Control de calidad	umbrales que eliminan poblaciones legítimas	Capítulo 5
Integración de varias muestras	lote confundido con la condición de interés	sesión 07

La columna de la derecha es, en la práctica, el mapa del módulo.

1.3.4. Tecnologías por principio de captura

Las plataformas se agrupan mejor por **cómo separan una célula de otra** que por fabricante.

	Gota	Placa	Indexado combinatorial
Ejemplos	10x Chromium 3 y 5	Smart-seq2, Smart-seq3	sci-RNA-seq
Cómo separa	una célula por gota, con código de barras	una célula por pozo	rondas sucesivas de códigos, sin aislar
Escala típica	miles a decenas de miles de células	cientos	decenas a cientos de miles
Cobertura del transcrito	un extremo	longitud completa	un extremo
Costo por célula	bajo	alto	muy bajo
Sensibilidad por célula	media	alta	media a baja

El esquema de gota con códigos de barras e identificadores moleculares únicos está descrito en Zheng et al. (2017). Los protocolos de longitud completa, en Picelli et al. (2014) y Hagemann-Jensen et al. (2020). El indexado combinatorial, que prescinde de aislar físicamente cada célula, en Cao et al. (2017).

1.3.5. Qué mide y qué no mide cada química

Pregunta	3	5	Longitud completa
Qué genes expresa cada célula	sí	sí	sí
Qué isoforma usa Repertorio inmune TCR/BCR	no	no	sí
Variantes en el transcrito	no	sí, con ensayo dirigido	parcial
Miles de células a costo bajo	solo cerca del extremo	solo cerca del extremo	sí
	sí	sí	no

La regla es que **la química se elige desde la pregunta**. Elegir 3 por costo y después querer isoformas o repertorio inmune no tiene solución bioinformática: la información no está en los datos.

1.3.6. Células íntegras o núcleos aislados

Esta decisión suele presentarse como una preferencia metodológica y no lo es: **la impone el tejido**.

	Células íntegras	Núcleos aislados
RNA que se recupera	citoplasmático y nuclear	nuclear, con alta proporción de intrones
Tejido congelado	inviable	viable
Tejido fibrótico o adiposo	rendimiento bajo y sesgado	viable
Neuronas, adipocitos, miocitos	se pierden o se fragmentan	se recuperan
Firma de estrés por disociación	frecuente	mínima
Se gana	más transcritos por célula	representatividad del tejido

La comparación sistemática entre ambos abordajes está en Ding et al. (2020), y su aplicación a tumores frescos y congelados en Slyper et al. (2020).

Lo importante para el módulo es que **el sesgo de composición no se detecta con métricas de calidad**. Una muestra en la que la disociación perdió las células más frágiles produce datos de excelente calidad técnica que describen el tejido equivocado. Solo se detecta comparando contra lo que se sabe del tejido; se retoma en el Capítulo 5.

1.3.7. Anatomía de una lectura de 10x

Cada lectura de una plataforma de gota lleva tres piezas de información:

Pieza	Dónde va	Qué responde
Código de barras de célula	lectura 1	de qué gota proviene la molécula
Identificador molecular único (UMI)	lectura 1	si es una molécula distinta o la misma amplificada
Inserto de cDNA	lectura 2	de qué gen proviene

Todas las moléculas capturadas en la misma gota comparten el **mismo código de barras**; cada molécula recibe su **propio UMI** antes de la amplificación. Al colapsar las lecturas que comparten código de barras, gen y UMI, se cuentan moléculas en lugar de copias de PCR.

De ahí la propiedad que se usará todo el módulo: **la matriz de *single-cell* contiene cuentas de moléculas**, no de lecturas. Dos consecuencias inmediatas:

- El UMI es único **dentro de una célula**, no en todo el experimento. Un mismo UMI en dos códigos de barras distintos corresponde a dos gotas distintas.
- Un código de barras **no es todavía una célula**. Puede ser una gota vacía con RNA ambiental, o dos células en la misma gota. Distinguirlos es el contenido del Capítulo 4 y del Capítulo 5.

1.3.8. Cuántas células y a qué profundidad

Más células no es automáticamente mejor. Con presupuesto de secuenciación fijo, aumentar el número de células reduce la profundidad por célula, y los genes de expresión baja dejan de detectarse. El compromiso se calcula a partir de la pregunta —qué población hay que detectar, con qué frecuencia esperada y con qué magnitud de efecto—; el tratamiento formal está en Svensson et al. (2017).

! La conclusión de la sesión

El peor desenlace de un experimento de célula única no es el que falla de forma visible. Es el que produce un resultado limpio que contesta una pregunta distinta de la que se quería hacer. Eso se previene en el diseño, no en el análisis.

1.4. Errores comunes

Error	Cómo se detecta	Cómo se resuelve
Elegir célula única cuando la pregunta era de <i>bulk</i>	Presupuesto gastado y potencia estadística insuficiente para comparar condiciones	Si la pregunta es “cambia la expresión entre dos condiciones” y no importa que tipo celular, el volumen con réplicas es más barato y más potente. Célula única se justifica cuando importa la composición o la heterogeneidad.
Usar células íntegras en tejido fibrótico o congelado	Rendimiento bajísimo, sesgo hacia las células más fáciles de disociar	Núcleos aislados. Se pierde el RNA citoplasmático pero se recupera la representatividad.
Elegir 3' y después querer isoformas o repertorio inmune	Los datos no contienen la información; no hay análisis que la recupere	La química se elige a partir de la pregunta, no al revés. Smart-seq para longitud completa; 5' con ensayos dirigidos para TCR y BCR.
Suponer que más células siempre es mejor	Muchas células con muy poca profundidad cada una; los genes poco expresados desaparecen	Hay un compromiso entre número de células y profundidad por célula. Se decide con la pregunta y se calcula, no se supone.

1.5. Recursos

Presentación; tabla comparativa de plataformas para completar en clase; fichas con los escenarios experimentales; documentación de químicas de 10x Genomics; artículo asignado como lectura previa; repositorio del curso en GitHub; rúbrica de evaluación; guía de acceso al clúster; Zoom.

En línea

Modalidad en línea Sesión en Zoom con pantalla compartida. La actividad de los tres escenarios se hace en un tablero de Miro, con un marco por equipo visible para todo el grupo. La tabla comparativa se llena ahí y queda como material de estudio. Entrega de cuentas del clúster por correo, con la tarea previa de acceso descrita abajo.

1.6. Si algo falla

Si falla	Plan B
Nadie leyó el artículo asignado	Llevar impresos el resumen y dos figuras; dar 10 minutos de lectura y sacrificar el momento Retar.

Si falla	Plan B
Las cuentas del clúster no están listas	No afecta esta sesión: es teórica. Se entregan por correo antes del 25 de agosto.

1.7. Referencias

Lectura de la sesión:

- Panorama del flujo completo y de sus decisiones: Luecken & Theis (2019).
- Plataforma de gota, códigos de barras y UMI: Zheng et al. (2017).
- Protocolos de longitud completa: Picelli et al. (2014), Hagemann-Jensen et al. (2020).
- Indexado combinatorial: Cao et al. (2017).
- Células íntegras frente a núcleos: Ding et al. (2020), Slyper et al. (2020).
- Número de células frente a profundidad: Svensson et al. (2017).

El texto de referencia del módulo, del que se toman las rutas de Bioconductor a partir del Capítulo 4, es Amezquita et al. (2020).

2 Diseño experimental

Sesión 02 · Jueves 20 de agosto de 2026 · sesión teórico-práctica, 120 minutos Imparten: Yalbi Itzel Balderas Martínez (90 min) y Luis Alberto Meza Cova (30 min)

i Objetivo

Diseñar un experimento de célula única que permita separar el efecto biológico del efecto técnico, aplicando los conceptos de réplica biológica, unidad de análisis, confusión entre variables y potencia estadística; y reconocer los diseños en los que ninguna corrección posterior puede rescatar el experimento.

2.1. Cómo se desarrolla la sesión

Clase teórico-práctica. Réplica biológica frente a réplica técnica, y por qué la célula no es la unidad de réplica: introducción a la agregación por muestra (pseudobulk) y al problema de la pseudorreplicación. Confusión entre lote y condición, con el caso límite en que el diseño es irrecuperable. Nociones de potencia: cuántas células y cuánta profundidad por célula. Estrategias de multiplexado (marcaje con anticuerpos, oligonucleótidos de multiplexado y demultiplexado genético) y qué problema resuelve cada una. Actividad: a cada equipo se le entrega un diseño experimental defectuoso, distinto para cada uno, que debe diagnosticar, rediseñar y defender ante el grupo.

- **6 bloques** de trabajo en 120 minutos, con descansos entre ellos
- Se alterna explicación y trabajo propio

2.2. Caso de aplicación

Caso de aplicación: el estudio que no se pudo salvar: un diseño en el que todos los casos se procesaron en una corrida y todos los controles en otra. El efecto de lote y el efecto de enfermedad son matemáticamente indistinguibles: no existe método de corrección que los separe, porque no hay ninguna observación que comparta lote y difiera en condición. Es el ejemplo más útil del curso porque demuestra que hay errores que el análisis no arregla, y que el momento de evitarlos es antes de secuenciar. Este mismo conjunto de datos es el que se retoma el 15 de septiembre para trabajar efecto de lote, de modo que los estudiantes comprueban en la práctica lo que predijeron en agosto.

2.3. Desarrollo

2.3.1. El experimento que no se pudo salvar

Seis pacientes y seis controles. Todos los pacientes se procesaron en una corrida; todos los controles, en otra. El resto del estudio está bien hecho: buena calidad, buena profundidad, análisis cuidadoso.

No se puede concluir nada.

La razón es estructural. Para separar el efecto de la enfermedad del efecto de la corrida haría falta al menos una observación que **comparta corrida y difiera en condición** —un control procesado junto a los pacientes, o al revés—. No existe. Los dos efectos están superpuestos punto por punto: cualquier diferencia entre grupos es compatible con «la enfermedad cambió la expresión» y con «las dos corridas difieren», sin que nada en los datos permita distinguirlas.

Esto es lo que significa que dos variables estén **confundidas**: no que sea difícil separarlas, sino que la información necesaria para hacerlo nunca se midió. Ninguna herramienta de corrección de lote la puede inventar.

! Importante

El momento de evitar este error es **antes de secuenciar**. Es el argumento central de la sesión y la razón por la que el diseño experimental va antes que cualquier método de análisis.

2.3.2. Réplica biológica, réplica técnica y la trampa de la célula

Una **réplica biológica** es una unidad experimental independiente: otro donante, otro ratón, otro cultivo iniciado por separado. Una **réplica técnica** es la misma unidad medida dos veces: dos alícuotas del mismo tejido, dos corridas de la misma biblioteca.

La distinción importa porque solo las réplicas biológicas informan sobre la variabilidad de la población, que es la que permite generalizar.

En *single-cell* aparece una tercera figura que no existe en *bulk*: **la célula**. La matriz tiene una célula por columna, y la tentación de tratar cada columna como una observación independiente es casi irresistible. No lo son: todas las células de un donante comparten su genotipo, su edad, su tratamiento, su tiempo de isquemia y su lote de procesamiento.

Tratarlas como independientes es **pseudorreplicación**, y su consecuencia es inmediata: el tamaño de muestra pasa de seis donantes a decenas de miles de células, los errores estándar se hunden y prácticamente cualquier diferencia resulta «significativa». El síntoma clásico es una lista de miles de genes diferenciales con valores de *p* extraordinariamente pequeños (Squair et al., 2021).

⚠ Una sola muestra con 10 000 células sigue siendo $n = 1$

Ese experimento describe a **una persona**. No hay análisis que convierta la variabilidad entre células de un individuo en variabilidad entre individuos.

2.3.3. Qué se hace en su lugar

Dos abordajes, según la pregunta:

Estrategia	Qué hace	Cuándo se usa
Pseudobulk	suma las cuentas de todas las células de una muestra —por tipo celular— y compara muestras entre sí	opción predeterminada para comparar condiciones
Modelos mixtos	modela al donante como efecto aleatorio y conserva las células como observaciones anidadas	cuando interesa la variabilidad dentro de la población

El pseudobulk provoca una objeción razonable: si se van a sumar las células, ¿para qué se hizo *single-cell*? La respuesta es que la resolución de célula única se usó para **decidir qué se está comparando**. El pseudobulk se calcula por tipo celular, de modo que la comparación es «linfocitos T de pacientes contra linfocitos T de controles» y no «tejido contra tejido». Eso es exactamente lo que el RNA-seq *bulk* no permite hacer, y es la misma lección del Capítulo 1: lo que cambió en aquel caso fue la composición, no la regulación.

2.3.4. Confusión entre variables

Un diseño está confundido cuando el efecto de interés y un efecto técnico varían juntos. El caso del inicio es el extremo, pero hay grados:

Diseño	Lote 1	Lote 2	¿Se pueden separar?
Confundido por completo	6 pacientes	6 controles	no
Equilibrado	3 pacientes + 3 controles	3 pacientes + 3 controles	sí
Parcialmente confundido	5 pacientes + 1 control	1 paciente + 5 controles	mal, pero es posible

La prevención es sencilla de enunciar y fácil de olvidar: **repartir las condiciones entre los lotes desde el diseño**. Si el experimento ya se corrió confundido, lo que corresponde es reportar la limitación, no aplicar una corrección que aparenta resolverla.

Hay una señal de alarma que conviene interiorizar: cuando el análisis de componentes principales muestra una **separación perfecta** entre los grupos, el primer impulso es celebrar. Un resultado demasiado limpio es indistinguible de un artefacto de lote total, y la primera pregunta debería ser cómo se repartieron las muestras entre corridas.

Lo mismo aplica a las covariables que no son de interés. El sexo, la edad, el sitio de colecta y el tiempo entre extracción y procesamiento deben **registrarse desde el inicio**, aunque no se planeen usarlos: no se pueden añadir después, y sin ellos un efecto inesperado queda sin explicación posible.

2.3.5. Potencia: tres cosas que compiten por el mismo presupuesto

Detectar un cambio en una población celular exige tres condiciones simultáneas: muestras suficientes para tener potencia estadística, células suficientes de esa población para que aparezca representada, y profundidad suficiente por célula para que el gen se detecte.

Con presupuesto fijo, las tres compiten:

Si se aumenta...	se paga con...	y se pierde
número de muestras	células por muestra	poblaciones poco frecuentes
células por muestra	profundidad por célula	genes de expresión baja
profundidad por célula	número de muestras	potencia estadística

La regla práctica, contraintuitiva para quien viene de pensar en términos de «más células es mejor»: **si el presupuesto no alcanza, se recorta el número de células antes que el número de muestras**. Tres réplicas biológicas por condición es el mínimo practicable.

Vale la pena hacer la aritmética de una población poco frecuente. Si la población de interés representa el 5 % del tejido, de 1 000 células capturadas se esperan unas 50; de 10 000, unas 500. Y eso por muestra. Con una población del 1 %, los mismos números se dividen entre cinco. La población rara se queda sin células mucho antes de lo que la intuición sugiere.

El tratamiento formal de este cálculo —qué tamaño de efecto se quiere detectar, con qué frecuencia esperada y con qué potencia— está en Svensson et al. (2017).

2.3.5.1. El cálculo con scPower

La aritmética anterior dice **cuántas células** salen de una población del 5 %. No dice si esas células **alcanzan para detectar el cambio**, que es otra pregunta y depende de tres cosas más: el tamaño del efecto que se quiere ver, la profundidad por célula y la potencia con la que uno se conforma.

scPower (Schmid et al., 2021) resuelve esa pregunta para estudios de varias muestras. Su utilidad real no es el número que devuelve, sino que **obliga a escribir los supuestos**: sin declarar frecuencia esperada de la población, tamaño de efecto, profundidad y número de muestras, no hay resultado.

Ahí está el punto que conviene retener:

Un número de células **sin sus supuestos** no se puede discutir ni defender. Es lo mismo que copiar un umbral de un tutorial.

Dos personas con supuestos distintos obtienen números distintos del mismo programa, y **las dos tienen razón**. Lo que distingue un diseño defendible de uno improvisado no es haber corrido scPower: es poder decir de dónde salió cada supuesto que se le metió.

Por eso, en el rediseño que se entrega como tarea, cualquier número de células que se proponga va acompañado de los supuestos que lo producen. No se evalúa acertar el número; se evalúa que el número sea rastreable.

i Cómo se ve en la práctica

scPower está fijado en el archivo de entorno del curso, así que se puede ejecutar, pero **no hace falta instalarlo para entender el argumento**. La demostración en vivo se retiró de la sesión del 20 de agosto por falta de tiempo; lo que importa —que el número depende de supuestos declarados— se sostiene sin correr una sola línea.

i El diseño y la química son una sola decisión

La plataforma elegida en el Capítulo 1 ya fijó el rango posible de células y de profundidad, y el tipo de preparación fijó qué poblaciones se pueden recuperar. No es «primero el diseño y luego la plataforma»: son dos caras de la misma decisión.

2.3.6. Multiplexado

Multiplexar es procesar **varias muestras en la misma corrida**, etiquetadas de forma que después se puedan separar.

Estrategia	Cómo etiqueta	Qué requiere
Marcaje con anticuerpos	anticuerpos con código de barras contra proteínas de superficie	que los anticuerpos reconozcan el tejido
Oligonucleótidos de multiplexado	oligonucleótidos con código anclados a la membrana	que la membrana los admita
Demultiplexado genético	los propios polimorfismos de cada donante	donantes genéticamente distintos

Lo que resuelve es preciso y vale la pena enunciarlo con cuidado:

- Permite poner **varias condiciones en el mismo lote**, con lo que el diseño deja de estar confundido por construcción.
- Reduce el número de corridas necesarias, de modo que el mismo presupuesto alcanza para **más muestras**.
- Hace **detectables** los dobletes formados entre muestras distintas, porque llevan dos etiquetas.

Y lo que no resuelve:

- **No crea réplicas biológicas**. Un diseño con una sola muestra por condición sigue siendo inservible aunque se multiplexe.
- El demultiplexado genético **no separa muestras del mismo donante** —dos tiempos, dos tratamientos sobre las mismas células—, porque genéticamente son idénticas.

Cada estrategia introduce además un supuesto nuevo. Que exista una técnica no obliga a usarla: un diseño que argumenta por qué no la necesita está tan bien defendido como uno que la adopta.

2.4. Errores comunes

Error	Cómo se detecta	Cómo se resuelve
Tratar cada célula como una réplica independiente	Valores de p absurdamente pequeños; miles de genes “significativos”	Agregar por muestra (pseudobulk) antes de comparar condiciones, o usar modelos mixtos. La réplica es el donante, no la célula.
Confundir lote con condición	Separación perfecta entre grupos en el PCA, que parece un gran resultado	Distribuir las condiciones entre lotes desde el diseño. Si ya ocurrió, se reporta la limitación; no se corrige.
Una sola muestra por condición	Resultados no generalizables; se está describiendo a dos individuos	Mínimo tres réplicas biológicas por condición. Si el presupuesto no alcanza, se reduce el número de células por muestra antes que el número de muestras.
Ignorar el sexo, la edad o el sitio de colecta	Efectos que aparecen después y no se pueden atribuir	Registrar todas las covariables desde el inicio, aunque no se vayan a usar. Añadir las después es imposible.

2.5. Recursos

Presentación; fichas con los diseños experimentales defectuosos; scPower en demostración para el cálculo de potencia; ejemplos de diseños publicados, acertados y fallidos; pizarra.

En línea

Modalidad en línea Los diseños defectuosos se reparten como tarjetas en Miro, un marco por equipo. Veinte minutos en salas simultáneas para rediseñar. La plenaria ocurre sobre el tablero: los tres equipos que exponen mueven sus propias tarjetas, de modo que la discusión sea visible y no solo hablada.

2.6. Si algo falla

Si falla	Plan B
Falla la demostración de scPower	Capturas de pantalla del resultado, preparadas el 19 de agosto. La demostración dura diez minutos y no vale la pena defenderla en vivo.
La plenaria se alarga	Exponen solo tres equipos; los otros entregan por escrito y reciben devolución.

2.7. Referencias

- Pseudorréplica y falsos descubrimientos en expresión diferencial de célula única: Squair et al. (2021). Es la lectura central de esta sesión.
- Número de células, profundidad y potencia: Svensson et al. (2017).
- Panorama general de decisiones del flujo: Luecken & Theis (2019).

i El arco hasta el 15 de septiembre

La predicción que cada equipo escribe hoy se reabre en el sesión 07, con los datos de ese mismo diseño. El razonamiento de por qué el conjunto de datos es el mismo está en [okf/analyses/decision-dataset-unico.md](#).

Parte II

Del FASTQ a un objeto en el que se puede confiar

3 De FASTQ a matriz

Sesión 03 · Martes 25 de agosto de 2026 · sesión práctica (bloque continuo, día 1), 120 minutos Imparten: Luis Alberto Meza Cova (30 min) y José Antonio Ovando Ricárdez (90 min), con Yalbi Itzel Balderas Martínez en los comandos y el acceso al clúster

i Objetivo

Reconstruir el camino que va de las lecturas crudas a la matriz de cuentas, comprendiendo qué hace un cuantificador y por qué contar moléculas no es contar lecturas; y ejecutar ese proceso en un entorno de cómputo de alto rendimiento siguiendo buenas prácticas.

3.1. Cómo se desarrolla la sesión

Clase teórico-práctica en el clúster, con envío de trabajos al gestor de colas. Puesta a punto del acceso. Inspección de las lecturas crudas desde la línea de comandos: localizar el código de barras de célula y el identificador molecular único, y estimar la profundidad de secuenciación. Qué hace un cuantificador: alineamiento o pseudoalineamiento, corrección de códigos de barras y deduplicación por identificador molecular. Panorama comparado de cuantificadores (Cell Ranger, STARsolo, alevin-fry y kallisto | bustools): qué supone cada uno y en qué difieren sus matrices. Actividad: enviar un trabajo real de cuantificación sobre un subconjunto de las lecturas e interpretar la salida del gestor de colas.

- **7 bloques** de trabajo en 120 minutos, con descansos entre ellos
- Se alterna explicación y trabajo propio

3.2. Caso de aplicación

Caso de aplicación: el trabajo que tarda ocho horas: la cuantificación completa de una corrida de 10x con Cell Ranger tarda horas y consume decenas de gigabytes de memoria. Esta sesión se diseñó alrededor de ese hecho, no a pesar de él: el trabajo se envía al gestor de colas en el minuto 60 y se recoge en la sesión siguiente. Los estudiantes aprenden así el patrón real de trabajo en un clúster, que es enviar y esperar, en lugar del patrón de escritorio, que es ejecutar y mirar. De paso se practica leer la salida del gestor de colas, que es donde aparecen los errores de memoria y de tiempo excedido.

3.3. Desarrollo

3.3.1. De dónde sale un número de la matriz

Toda la sesión cabe en una pregunta sobre una matriz de cuentas ya hecha: en la fila de *CD3E* y la columna de una célula hay un **7**. ¿De dónde salió?

La respuesta corta es que ese 7 no son siete lecturas: son **siete moléculas distintas** de *CD3E* que había en esa célula. Detrás pudo haber cientos de lecturas, porque la PCR amplifica. Entre las lecturas crudas y ese número hay una cadena de decisiones que conviene conocer, porque todo el análisis posterior del módulo se apoya en esa matriz.

3.3.2. Qué hace un cuantificador

Cualquier cuantificador de célula única resuelve cuatro problemas, en este orden:

1. **Ubicar la lectura** en el genoma o el transcriptoma, por alineamiento o por pseudoalineamiento. Esto define la **fila** de la matriz.
2. **Corregir el código de barras de célula**, para saber de qué gota vino. Define la **columna**.
3. **Deduplicar por identificador molecular único (UMI)**, para contar moléculas y no copias de PCR.
4. **Contar**, y escribir la matriz.

Los cuatro pasos están en todas las herramientas. Lo que cambia entre ellas es cómo resuelve cada una, y ahí es donde aparecen las diferencias entre matrices.

3.3.2.1. Alinear o pseudoalinear

El alineamiento busca la posición exacta de la lectura en el genoma; el pseudoalineamiento se conforma con saber a qué transcritos es compatible.

	Alineamiento	Pseudoalineamiento
Herramientas	Cell Ranger, STARsolo (Dobin et al., 2013; Kaminow et al., 2021)	alevin-fry (He et al., 2022), kallisto bustools (Melsted et al., 2021)
Costo	más tiempo y memoria	mucho más rápido
Permite	variantes, splicing, intrones	conteo por gen

Para el conteo por gen, que es lo que necesita este módulo, los resultados son muy parecidos. Las diferencias se concentran en familias de genes parálogos y en los bordes de los transcritos.

3.3.2.2. Corregir el código de barras

La secuenciación comete errores. Un código de barras leído con una base cambiada parece una gota nueva que nunca existió, y sin corrección aparecerían miles de «células» falsas con muy pocas cuentas cada una.

La corrección compara cada código observado contra la **lista de códigos posibles** de la química empleada: si difiere en una base de uno abundante, se corrige hacia él; si no se parece a ninguno, se descarta. Este paso es la razón de que el número de códigos de barras observados y el número de células reales no coincidan, algo que se vuelve central en el Capítulo 4.

3.3.2.3. Contar moléculas, no lecturas

Aquí está la idea que da nombre a la sesión. Dos lecturas que comparten **el mismo código de barras, el mismo UMI y el mismo gen** provienen de la misma molécula original: una fue amplificada a partir de la otra. Se colapsan a una sola cuenta.

! Importante

Sin deduplicación, los genes muy amplificados quedarían sistemáticamente sobrestimados, y la magnitud del sesgo dependería de cuántos ciclos de PCR se hicieron —es decir, de un detalle técnico y no de la biología.

Esto explica también por qué un conjunto de datos **sin UMI** —Smart-seq2, por ejemplo— se interpreta distinto: ahí lo que se cuenta son lecturas, y la normalización tiene que corregir además por la longitud del transcrito.

3.3.3. Los archivos que llegan de la secuenciación

Una corrida de 10x entrega los datos repartidos en archivos con una convención de nombres que **el cuantificador usa para saber cuál es cuál**:

Archivo	Qué contiene	Para qué sirve
..._R1_...fastq.gz	código de barras de célula + UMI	columna de la matriz y deduplicación
..._R2_...fastq.gz	el inserto, la secuencia del transcrito	fila de la matriz
..._I1_...fastq.gz	índice de muestra, si viene	demultiplexado entre muestras

La lectura 1 es notablemente más corta que la lectura 2, y esa asimetría es la misma que se dibujó sobre papel en el Capítulo 1: el código de barras y el UMI ocupan unas decenas de bases, mientras que el inserto ocupa el resto.

Advertencia

Renombrar un FASTQ «para ordenar» rompe la corrida. Los sufijos R1, R2 e I1 no son decorativos.

3.3.4. Trabajar en un clúster: enviar y esperar

La cuantificación completa de una corrida de 10x tarda horas y consume decenas de gigabytes de memoria. La sesión está diseñada alrededor de ese hecho: el trabajo se envía al gestor de colas y se recoge en la sesión siguiente.

Eso no es una limitación del curso, es **el patrón real de trabajo** en cómputo de alto rendimiento. En un escritorio uno ejecuta y mira; en un clúster uno envía y espera, y mientras tanto hace otra cosa.

Tres reglas que conviene interiorizar desde el primer día:

- **Siempre sbatch, nunca en el nodo de acceso.** Correr una cuantificación en el nodo de acceso lo satura para todo el mundo. Es una norma de convivencia antes que una buena práctica técnica.
- **Pedir margen de memoria y de tiempo.** Si la memoria se queda corta, el trabajo muere sin una explicación clara varias horas después. La causa real queda registrada en el archivo de salida del gestor, que es el primer lugar donde hay que mirar.
- **Anotar el identificador del trabajo.** Es lo que permite consultarlo después, y sin él no se sabe cuál de los trabajos en cola es el propio.

3.3.5. Los comandos, de principio a fin

Esto es lo que se ejecuta en la práctica, en orden. Está aquí y no solo en la libreta para que quede en el material de consulta: quien repita la sesión meses después no va a abrir el notebook, va a abrir el manual.

1. Cargar el entorno. Una sola línea, cada vez que se entra al clúster.

```
source /mnt/data/bioinfo3/compartido/setup-curso.sh
```

Define `$REFERENCIA` —la referencia GRCh38— y `$DATOS`. Conviene ver qué es `$DATOS` exactamente antes de usarlo:

```
echo "$DATOS"
ls "$DATOS"
```

`$DATOS` apunta **ya** a la carpeta del conjunto de datos, no al directorio que la contiene. Dentro están las dos bibliotecas —`gex_1` y `gex_2`— con su `CHECKSUMS.md5` y su `PROCEDENCIA.md`.

2. Mirar los archivos antes de cuantificar.

```

BIBLIOTECA=$DATOS/manual_5k_pbmc_NGSC3_ch5_gex_1
TRABAJO=/mnt/data/bioinfo3/$USER/sesion03
mkdir -p "$TRABAJO"
cd "$TRABAJO"

ls -lh "$BIBLIOTECA"

```

Deben aparecer seis archivos —R1, R2 e I1 por cada uno de los dos carriles, L001 y L002—, unos 7.7 GB en total. Si la lista sale vacía o da un error de directorio, hay que parar ahí: nada de lo que sigue va a funcionar.

❗ El directorio de trabajo no es \$HOME

La cuota de \$HOME es de 5 GB y la salida de Cell Ranger no cabe. Por eso \$TRABAJO apunta a /mnt/data/bioinfo3/\$USER/, que es el espacio propio de cada quien dentro del volumen del curso.

3. Escribir el guion del trabajo.

```

cat > cuantificar.sh <<'FIN'
#!/usr/bin/env bash
#SBATCH --job-name=cuant-pbmc
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=04:00:00
#SBATCH --output=cuant-%j.out
#SBATCH --error=cuant-%j.err
set -euo pipefail

source /mnt/data/bioinfo3/compartido/setup-curso.sh

cd /mnt/data/bioinfo3/$USER/sesion03
cellranger count --id=pbmc_5k --transcriptome=$REFERENCIA --fastqs=$DATOS/manual_5k_pbmc_NGSC3
FIN

```

⚠ El trabajo arranca en una máquina limpia

Ésta es la trampa que costó la práctica del 25 de agosto de 2026 a casi todo el grupo, y no dio ningún error entendible.

sbatch no hereda las variables de la terminal desde la que se envía. Dentro del guion **solo existen** las que él mismo define y las que trae **setup-curso.sh** —por eso esa línea está también aquí dentro, y no es una repetición descuidada—.

Una variable que no existe se expande a la cadena vacía, sin avisar. Si el guion dijera `--fastqs=$FASTQS/..._gex_1` y `$FASTQS` se hubiera definido fuera, Cell Ranger recibiría `--fastqs=/..._gex_1`: una ruta absoluta inexistente, con una barra inicial como única pista. El trabajo muere horas después, o de inmediato, con un mensaje sobre un directorio que nadie escribió.

La regla: dentro del guion, rutas completas o variables de `setup-curso.sh`. Nunca variables de la sesión de fuera.

4. Enviarle y seguirle la pista.

```
sbatch cuantificar.sh      # imprime el identificador del trabajo: anótalo
squeue -u "$USER"         # en qué va
```

Cuando termine, la salida queda en `$TRABAJO/pbmc_5k/outs/`, y los mensajes del trabajo en `cuant-<identificador>.out` y `.err`. El archivo `.err` se lee aunque todo haya salido bien: ahí aparecen los avisos que no detienen la corrida pero cambian cómo se interpreta.

Esa carpeta `outs/` es con lo que abre el Capítulo 4.

3.3.6. Cuatro cuantificadores y por qué no dan lo mismo

En clase se corre uno y se comparan los cuatro sobre resultados ya calculados. Lo que interesa no es cuál es «el mejor», sino qué supone cada uno:

Herramienta	Enfoque	Qué hay que saber
Cell Ranger	alineamiento, flujo cerrado de 10x	es el de referencia para datos de 10x; poco configurable
STARsolo	alineamiento con STAR	reproduce a Cell Ranger y es más rápido y configurable
alevin-fry	pseudoalineamiento selectivo	muy eficiente en memoria; decisiones explícitas sobre el conteo
kallisto bustools	pseudoalineamiento	el más rápido; formato intermedio propio

Las matrices resultantes **no tienen las mismas dimensiones**, y la razón principal no es el alineamiento sino el paso de decidir qué código de barras corresponde a una célula real. Esa decisión —el llamado de células— es precisamente el tema del Capítulo 4.

i La conclusión de la sesión

Elegir un cuantificador es aceptar sus supuestos. La matriz no es un dato objetivo que estaba ahí esperando: es el resultado de una cadena de decisiones, y conviene poder nombrarlas cuando alguien pregunte de dónde salió un número.

3.4. Errores comunes

Error	Cómo se detecta	Cómo se resuelve
Usar una referencia que no corresponde al organismo o a la versión de anotación	Porcentaje de mapeo bajísimo, o genes que no existen	Verificar la referencia antes de correr. El resumen web de Cell Ranger reporta el porcentaje de lecturas mapeadas: si está muy por debajo de lo esperado, se detiene y se revisa.
Correr el trabajo en el nodo de acceso en lugar de enviarlo a la cola	El nodo de acceso se satura y afecta a todo el clúster	Siempre sbatch. Es además la norma de convivencia del clúster, no solo una buena práctica técnica.
Pedir muy poca memoria al gestor de colas	El trabajo muere sin explicación clara horas después	Estimar la memoria antes y pedir margen. Leer el archivo de salida del gestor, que registra la causa real de la muerte del proceso.
Confundir lecturas con moléculas	Sobrestimación de la expresión de genes muy amplificados	La deduplicación por identificador molecular único existe precisamente por esto. Si el conjunto de datos no tiene UMI (Smart-seq2), el conteo se interpreta distinto.

3.5. Recursos

Clúster institucional con gestor de colas; Cell Ranger con referencia humana GRCh38; STARsolo, alevin-fry y kb-python; seqkit, FastQC y MultiQC; lecturas crudas de PBMC en el directorio compartido; guía de acceso al clúster; guion en Quarto provisto.

En línea

Modalidad en línea Sala principal para la teoría y salas simultáneas para el acceso al clúster, con un instructor por sala. Requisito previo obligatorio: cada estudiante debe haber entrado al clúster y subido una captura antes del 25 de agosto. En línea no se puede asomar uno a la pantalla del que se atoró, así que el acceso se resuelve antes, no durante.

3.6. Si algo falla

Si falla	Plan B
Un estudiante no logra entrar por SSH	Que trabaje en pareja. Su acceso se resuelve fuera del horario de clase, no en vivo.
El entorno de R y Python no está instalado	No hay plan B. Por eso la fecha límite del técnico es el lunes 24 de agosto.

Si falla	Plan B
El clúster está saturado y el trabajo no arranca	Reservación de partición para el horario de clase, solicitada al técnico. Si aún así falla, se usan las matrices precalculadas y la práctica se convierte en lectura de la salida de un trabajo ya corrido.

3.7. Referencias

- Plataforma de gota, códigos de barras y UMI: Zheng et al. (2017).
- STAR y su modo de célula única: Dobin et al. (2013), Kaminow et al. (2021).
- alevin-fry: He et al. (2022).
- kallisto | bustools: Melsted et al. (2021).
- Panorama general del flujo y sus decisiones: Luecken & Theis (2019).

El texto de referencia del módulo, con las rutas de Bioconductor que se usan a partir del Capítulo 4, es Amezquita et al. (2020).

4 De matriz a objeto listo. Rutas de importación y estructuras de datos

Sesión 04 · Jueves 27 de agosto de 2026 · sesión práctica (bloque continuo, día 2), 120 minutos Imparten: Yalbi Itzel Balderas Martínez (90 min) y Alfredo Morales Salazar (30 min)

i Objetivo

Manipular con soltura el objeto de datos de célula única; reconocer las distintas rutas por las que pueden llegar datos de célula única y qué estructura produce cada una; y distinguir gotas vacías de gotas con célula mediante criterios estadísticos y no por un umbral arbitrario.

4.1. Cómo se desarrolla la sesión

Práctica guiada con teoría intercalada. Recuperación e interpretación del trabajo enviado el martes. Por qué la matriz de expresión es dispersa y qué implica para la memoria. Anatomía del objeto SingleCellExperiment, y tabla de equivalencias con los objetos Seurat y AnnData. Rutas de importación: desde FASTQ, desde una matriz suelta, desde un objeto de práctica, desde GEO, desde CZ CELLxGENE Discover, desde un archivo .rds de un colaborador y desde un archivo .loom; qué estructura produce cada una y qué trampa trae. Llamado de células: curva de rango de códigos de barras y emptyDrops, contrastados con el umbral fijo del programa comercial. Actividad: a cada equipo se le entrega un conjunto de datos de origen distinto que debe importar y caracterizar antes de tocarlo.

- **6 bloques** de trabajo en 120 minutos, con descansos entre ellos
- Se alterna explicación y trabajo propio

4.2. Caso de aplicación

Caso de aplicación: el colaborador que manda un .rds: la situación más común en la vida real de un laboratorio: llega un archivo .rds por correo, sin documentación, y hay que averiguar qué contiene. Puede venir filtrado, normalizado, con reducciones ya calculadas y con una versión de Seurat distinta a la instalada. El ejercicio consiste en interrogar al objeto antes de analizarlo: cuántas células tiene, si los valores son enteros o decimales, si existe una capa de cuentas crudas, y qué versión lo generó. Es la habilidad que separa a quien puede colaborar de quien solo puede correr su propio guion.

4.3. Desarrollo

4.3.1. Dos matrices y una decisión

El trabajo enviado el martes deja una carpeta `outs/` con **dos** matrices, y la primera decisión de la sesión es cuál de las dos se usa:

	Qué contiene
<code>raw_feature_bc_matrix</code>	todos los códigos de barras observados
<code>filtered_feature_bc_matrix</code>	solo los que el programa llamó «célula»

No hay una respuesta universal, porque depende de lo que se vaya a hacer. Si el objetivo es analizar tipos celulares y se acepta el criterio del programa, **filtered** sirve. Si se va a hacer el llamado de células por cuenta propia, o se va a estimar RNA ambiental, hace falta **raw**.

Advertencia

filtered ya tiene aplicado un algoritmo que alguien más eligió, con parámetros que no se ven. Tomarla y luego intentar control de calidad de gotas no produce un error: simplemente no encuentra nada, porque ya se hizo.

4.3.2. Por qué la matriz es dispersa

Una célula expresa una fracción de los genes anotados, y de los transcritos presentes solo se captura una parte. El resultado es que la gran mayoría de las entradas de la matriz son ceros —típicamente más del 90 %.

Eso no es un defecto: es una propiedad de los datos que tiene consecuencias prácticas inmediatas. Una matriz de 36 000 genes por 10 000 células tiene 360 millones de entradas; almacenada de forma densa, a 8 bytes por valor, son unos 2.9 GB. Guardando solo los valores distintos de cero y sus posiciones, el mismo contenido ocupa alrededor de 0.2 GB.

Importante

as.matrix() sobre una matriz dispersa la convierte a densa y suele terminar la sesión de R. No se convierte a denso «para ver mejor»: para inspeccionar se toma una esquina.



Figura 4.1: Izquierda: la matriz de cuentas, con más del noventa por ciento de las casillas vacías. Derecha: lo que se guarda de verdad —la lista de las casillas con cuentas— y la comparación de memoria, a escala.

4.3.2.1. De dónde salen los ceros, y por qué no son datos faltantes

Un cero de esa matriz puede venir de dos sitios, y no significan lo mismo:

1. **El gen no se expresa en esa célula.** Eso es biología.
2. **Se expresa y no se capturó.** Eso es muestreo: de las moléculas que hay dentro de la célula, solo una fracción llega a convertirse en una cuenta. Cuánta, depende de la química, y vale la pena verlo con números.

Ninguno de los dos es un dato faltante. Un dato faltante es un valor que existe y no se midió. Aquí el valor sí se midió, y salió cero.

4.3.2.1.1. Cuántos transcritos se capturan de verdad

Las dos fuentes que se citan habitualmente **no dan el mismo número, porque no miden lo mismo**:

Fuente	Cifra	Qué mide
10x Genomics , su propia documentación (10x Genomics, s. f.)	6.7–8.1 % (v1) · 14–15 % (v2) · 30–32 % (v3)	fracción de transcritos de mRNA capturados por célula, por química
Svensson et al. 2017 (Svensson et al., 2017)	5–10 % para RNA endógeno, contra smFISH · 0.5–1 % para los ERCC	eficiencia molecular medida contra un patrón

Dos avisos antes de usar cualquiera de las dos cifras:

- **La química importa mucho.** De la v1 a la v3 la captura se cuadruplica. Este módulo trabaja con **v3**, así que el número que aplica es **alrededor del 30 %**, no el 10 % que suele citarse arrastrado de artículos anteriores.

- **Svensson mide el RNA endógeno un orden de magnitud mejor que los spike-ins** —5–10 % frente a 0.5–1 %— y advierte explícitamente que trasladar el límite de detección de los ERCC al mRNA real es dudoso: los ERCC entran en la reacción de otra manera.

Que la captura sea del 30 % y no del 100 % es, por sí sola, la explicación de buena parte de los ceros de la matriz.

4.3.2.1.2. El muestreo, con la matemática mínima

Si una célula tiene m moléculas de un gen y cada una se captura con probabilidad p , lo que se cuenta es una binomial. Y si m es a su vez Poisson con media λ , lo que sale al final **sigue siendo Poisson**, con media $p\lambda$.

Esa propiedad es cómoda y conviene decirla en voz alta: el muestreo no cambia la familia de la distribución, solo la escala. Por eso las cuentas se siguen comportando como cuentas después de capturarlas, y por eso los métodos las modelan directamente en lugar de «arreglarlas» antes.

De ahí sale la primera consecuencia: en una Poisson, **la media y la varianza son el mismo número**.

4.3.2.1.3. Los datos reales tienen más varianza que eso

Dos células del mismo tipo no expresan lo mismo. Si la media de cada célula varía siguiendo una distribución gamma, la mezcla de Poisson con gamma da una **binomial negativa**, que es Poisson más un parámetro para esa varianza de más:

$$\text{Var}(Y) = \mu + \frac{\mu^2}{\theta}$$

El primer término es el muestreo, que no se puede evitar. El segundo es la variabilidad biológica. Cuando θ crece, el segundo término desaparece y la binomial negativa vuelve a ser una Poisson. Es el modelo de trabajo de casi todo el análisis de expresión diferencial.

4.3.2.1.4. ¿Y hacen falta todavía más ceros?

Es la pregunta de la inflación de ceros, y tiene respuesta distinta según el protocolo. Para datos con identificador molecular único, la Poisson y la binomial negativa explican los ceros observados **sin necesidad de añadir un componente extra** (Svensson, 2020). Para protocolos de longitud completa, donde la amplificación por PCR mete su propia variabilidad, la discusión sigue abierta (Hicks et al., 2018).

4.3.2.1.5. Qué quiere decir «el supuesto se rompe», con un número

Se repite mucho que alimentar un método de conteos con valores ya transformados «rompe el supuesto». Conviene poder enseñarlo en vez de afirmarlo.

La medida es el **índice de dispersión**: la varianza dividida entre la media. Para una Poisson vale **exactamente 1** —media y varianza son el mismo número—, y ese valor es el **piso** de cualquier modelo de conteos: la varianza de una Poisson no puede ser menor que su media.

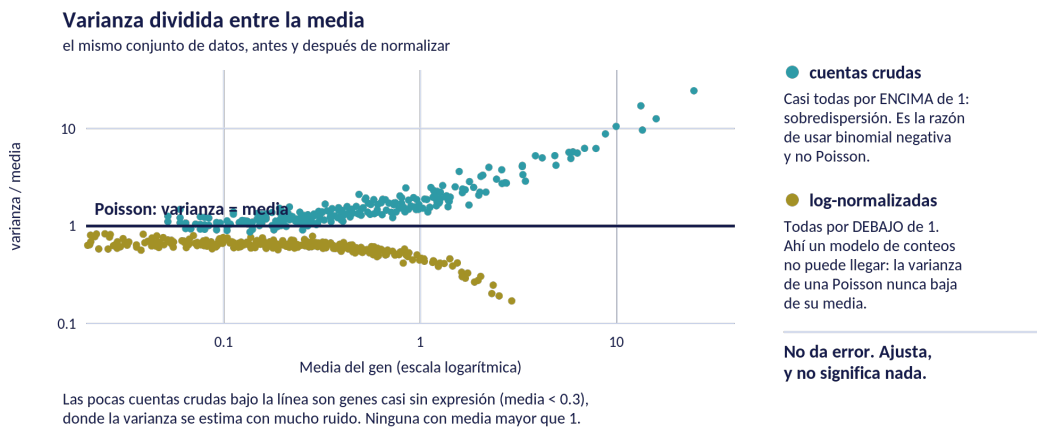


Figura 4.2: El mismo conjunto de datos antes y después de normalizar. La línea marca el valor 1, el de una Poisson. Las cuentas crudas quedan por encima; las log-normalizadas, todas por debajo.

Sobre una simulación de 300 genes y 250 células, el resultado es nítido:

	Índice de dispersión (genes con media > 1)
Cuentas crudas	mediana 2.49 , entre 1.4 y 24
Log-normalizadas	mediana 0.38 , entre 0.17 y 0.47

Las crudas caen **por encima** de 1: eso es la sobredispersión, y es exactamente la razón por la que se usa binomial negativa y no Poisson. Las transformadas caen **todas por debajo** —280 de 280—, en una zona que **ningún modelo de conteos puede representar**.

Ahí está la rotura: al alimentar un modelo de conteos con valores transformados, **el modelo ve una varianza menor que su propio mínimo**. No da error. Ajusta, y lo que sale no significa nada.

4.3.2.1.5.1. Por qué el eje vertical es un cociente y no la varianza

Parece un detalle de presentación y no lo es: **sin esa división, la figura no se puede dibujar**. Hay tres razones, y conviene verlas con dos genes concretos de la propia simulación.

	Media	Varianza	Varianza / media
Gen poco expresado, crudas	0.132	0.147	1.11

	Media	Varianza	Varianza / media
Gen muy expresado, crudas	13.488	129.833	9.63

1. El cociente quita lo que ya sabíamos. La varianza del gen muy expresado es **882 veces** la del otro. Ese número no dice nada interesante: lo que más se expresa, más varía, y eso es aritmética, no biología. Al dividir entre la media, la diferencia se reduce a **8.6 veces**, y *esa* sí es informativa — mide cuánto más varía cada gen **de lo que le tocaría**.

2. El cociente vuelve horizontal la referencia. La hipótesis nula es Poisson: varianza = media. Graficando varianza contra media, esa nula es una **diagonal**, y decidir si un punto cae encima o debajo de una diagonal a lo largo de cuatro décadas es incómodo — el ojo humano es malo para eso. Dividido, la nula se convierte en una **horizontal en 1** y la pregunta se contesta de un vistazo.

Es exactamente la misma razón por la que en expresión diferencial se usa el gráfico MA en vez de graficar una condición contra la otra: se rota el problema para que lo esperado quede plano y lo interesante salte.

3. El cociente hace comparables dos escalas que no lo son — y esta es la decisiva para esta figura. Los mismos dos genes, después de normalizar:

	Media	Varianza	Varianza / media
Gen poco expresado, log-normalizada	0.082	0.057	0.70
Gen muy expresado, log-normalizada	2.350	0.582	0.25

Las varianzas crudas llegan a **129**; las log-normalizadas no pasan de **0.6**. En un eje de varianza, las dos nubes viven en escalas incomparables y no caben en el mismo gráfico. El cociente **no tiene unidades**, y por eso se pueden mirar juntas.

💡 El matiz que la propia figura enseña

El cociente **no aplana del todo** la nube de las cuentas crudas: sube hacia la derecha, de 1.11 en el gen poco expresado a 9.63 en el muy expresado. Eso es correcto y no un defecto.

Para una binomial negativa, el cociente vale

$$\frac{\text{Var}(Y)}{\mu} = \frac{\mu + \mu^2/\theta}{\mu} = 1 + \frac{\mu}{\theta}$$

es decir, **la división quita la parte de Poisson del crecimiento y deja la sobredispersión**, que sí depende de la media. Si los datos fueran Poisson pura, θ sería infinito y la nube quedaría **plana en 1**.

Que suba es la sobredispersión hecha visible, y su pendiente informa de θ . Es una buena pregunta para el grupo: *¿por qué la nube de las crudas no es plana?* — separa a quien entendió el cociente de quien solo lo leyó.

El cociente tiene nombre propio, **índice de dispersión** o *factor de Fano*, y su lectura es directa:

Valor	Qué significa	Dónde se ve
$= 1$	Poisson. Toda la variación es de muestreo	la línea de la figura
> 1	sobredispersión: varía más de lo que el muestreo explica	las cuentas crudas
< 1	subdispersión	los valores transformados — y ningún modelo de conteos puede representarlo

i Una hipótesis que la simulación descartó

La primera versión de esta figura iba a mostrar que la transformación **destruye** la relación entre media y varianza. Al simular los datos antes de dibujarlos resultó **falso**: la correlación entre el logaritmo de la media y el de la varianza es 0.990 en los dos casos, y los puntos siguen alineados. Lo que cambia no es que la relación desaparezca, sino **dónde queda respecto del piso de Poisson**.

Se deja anotado porque es la explicación equivocada más natural, y porque el guion que genera la figura imprime las cifras para que cualquiera las contraste.

! Qué se sigue de esto en la práctica

- **No se imputan los ceros por costumbre.** Imputar supone que falta un dato, y no falta. Es una decisión que hay que poder defender, no un paso de rutina.
- **Los métodos piden cuentas.** Darles valores ya transformados rompe la relación entre media y varianza sobre la que están contruidos.
- **Guardar solo lo distinto de cero no pierde nada:** los ceros siguen ahí, simplemente no se escriben.

4.3.3. Anatomía del objeto

Un `SingleCellExperiment` guarda la matriz y todo lo que se sabe de sus filas y columnas en un mismo objeto:

Ranura	Qué guarda
<code>assays</code>	las matrices: <code>counts</code> , <code>logcounts</code> , y las que se añadan
<code>rowData</code>	metadatos de genes, una fila por gen
<code>colData</code>	metadatos de células: lote, donante, condición
<code>reducedDims</code>	representaciones reducidas: PCA, UMAP
<code>metadata</code>	información que no encaja en lo anterior

La ventaja no es organizativa sino de corrección: al filtrar células se filtran a la vez las columnas del assay y las filas de `colData`, de modo que no pueden desalinearse. Mantener los metadatos en un `data.frame` aparte es la manera clásica de acabar analizando una tabla de condiciones que ya no corresponde a las células que quedaron.

4.3.3.1. Cómo se guarda y cómo se imprime, que no es lo mismo

Conviene separar dos cosas que se confunden con facilidad.

Cómo se guarda. Los tres objetos usan almacenamiento disperso con la misma idea: solo los valores distintos de cero, cada uno con su fila y su columna. En eso son equivalentes, y la matriz dispersa es una pieza que vive **dentro** del objeto, no una alternativa a él.

Cómo se imprime. Ahí sí difieren. En R, tanto `SingleCellExperiment` como `Seurat` guardan la matriz como un `dgCMatrix` del paquete `Matrix`, y esa clase imprime los ceros como puntos:

```
counts(sce)[1:4, 1:6]
#> 4 x 6 sparse Matrix of class "dgCMatrix"
#> GEN1  . . 3 . . .
#> GEN2  1 . . . 2 .
#> GEN3  . . . . . .
#> GEN4  . 5 . . . 1
```

Los puntos **no son de Seurat**: son de `Matrix`, y por eso salen igual en un `SingleCellExperiment`. En Python, `AnnData` usa `scipy.sparse`, que no imprime puntos: hay que pedir `.toarray()` sobre una esquina para ver los valores.

Es una diferencia de presentación, no de contenido — pero explica por qué el mismo dato «se ve distinto» según desde dónde se mire.

4.3.3.2. Los otros dos objetos

	SingleCellExperiment	Seurat	AnnData
Lenguaje	R	R	Python
Cuentas	<code>counts(sce)</code>	<code>obj[["RNA"]]\$counts</code>	<code>adata.X</code> o <code>adata.raw</code>
Metadatos de célula	<code>colData</code>	<code>obj@meta.data</code>	<code>adata.obs</code>
Metadatos de gen	<code>rowData</code>	<code>obj[["RNA"]]@meta.data</code>	<code>adata.var</code>
Reducciones	<code>reducedDims</code>	<code>obj@reductions</code>	<code>adata.obsm</code>
Orientación	genes × células	genes × células	células × genes

La trampa de la orientación

`AnnData` está transpuesto respecto de los otros dos. Al convertir entre objetos las dimensiones se intercambian, y si no se verifica se puede terminar analizando una matriz al revés sin ningún mensaje de error.

Después de cualquier conversión conviene comprobar tres cosas: las dimensiones, la suma total de cuentas y que los metadatos de lote sobrevivieron. Ninguna de las tres avisa cuando falla.

4.3.4. Las siete rutas de importación

Casi nunca se empieza desde los FASTQ. Se empieza desde lo que haya, y quien solo conoce una ruta se paraliza la primera vez que le llega otra. La tabla completa, con el detalle de cada una, está en el [anexo de rutas de importación](#); aquí interesa el patrón: **cada ruta trae una trampa característica y ninguna avisa con un error.**

- **Cell Ranger.** La disyuntiva entre **raw** y **filtered**.
- **GEO.** Lo que subió el autor. Puede ser cuentas, pero también TPM, CPM o una matriz ya normalizada, con frecuencia sin decirlo en ninguna parte.
- **CZ CELLxGENE.** El repositorio público de la Chan Zuckerberg Initiative. Reúne conjuntos de célula única ya publicados y los **re-anota con un esquema común**: tipo celular, tejido, enfermedad y ensayo con términos de ontología, para que se puedan comparar entre estudios. Esa misma estandarización explica la trampa: lo que se deposita ahí suele estar ya procesado, así que X trae valores normalizados y las cuentas, cuando están, viven en **raw**.
- **Un colaborador.** Un **.rds** que puede venir filtrado, normalizado, y serializado con otra versión de Seurat.

i Una regla que resuelve la mayoría de los casos

Si hay decimales, no son cuentas crudas. Un conteo de moléculas es un entero.

Esto importa más de lo que parece: los métodos estadísticos de célula única suponen conteos —modelos de Poisson, binomial negativa—. Alimentarlos con valores ya transformados no produce un error, produce resultados que parecen correctos y no lo son.

4.3.4.1. Un detalle al unir muestras

Antes de nada, qué quiere decir «unir». Un estudio rara vez tiene una sola muestra: hay varios donantes, varias condiciones o varias corridas, y cada una llega en su propia carpeta y se importa por separado. Para compararlas hay que combinarlas en **un solo objeto**, y eso es unir:

SingleCellExperiment	<code>cbind(sce1, sce2)</code>
Seurat	<code>merge(obj1, obj2)</code>
AnnData	<code>ad.concat([a1, a2], keys = ["m1", "m2"])</code>

Ocurre **después de importar y antes de comparar nada**. En este módulo les toca el 1 de septiembre, al juntar los dos lotes de la práctica de control de calidad, y otra vez el 15 de septiembre con el conjunto multi-lote.

Los códigos de barras se repiten entre muestras, y conviene entender por qué y no solo recordarlo.

La lista de códigos de barras de 10x es fija y pública. No se inventa una secuencia por célula: el fabricante fabrica las perlas con secuencias tomadas de una lista cerrada, distinta para cada química —**737 280** secuencias en la v2 y **6 794 880** en la v3—, y cada corrida reparte sus células entre esas mismas secuencias. Es lo mismo que hace posible la corrección de códigos de barras que se vio el martes: se puede comparar contra la lista porque la lista existe de antemano.

La consecuencia es inmediata: **dos muestras distintas usan el mismo repertorio de nombres**. AAACCCAAGCGTAAGA-1 existe en prácticamente todas, y no son la misma célula.

Conviene mirarlo en el objeto en vez de imaginarlo:

```
head(colnames(sce), 3)
#> "AAACCCAAGCGTAAGA-1" "AAACCCACATCAGTCA-1" "AAACCCAGTGTAACGG-1"

colnames(sce) <- paste0("muestra1_", colnames(sce))

head(colnames(sce), 3)
#> "muestra1_AAACCCAAGCGTAAGA-1" "muestra1_AAACCCACATCAGTCA-1" "muestra1_AAACCCAGTGTAACGG-1"
```

Son 16 nucleótidos, que dan 4^{16} —más de cuatro mil millones— de combinaciones posibles. La lista de 10x usa una fracción pequeña de ellas, elegidas de antemano y bien separadas entre sí: por eso una lectura con un error de una base todavía se puede asignar sin ambigüedad al código correcto.

El sufijo tampoco ayuda. El **-1 no identifica la muestra**: numera el pocillo dentro de esa corrida, y casi siempre vale 1. Quien lo lea como un identificador de muestra se lleva una sorpresa al unir.

Al combinar sin prefijar el identificador de muestra, dos células distintas comparten nombre y **colisionan**. El objeto resultante tiene menos células de las que debería, sin que nada lo señale.

```
colnames(sce) <- paste0("muestra1_", colnames(sce))
```

4.3.5. Las cuatro preguntas obligatorias

Antes de normalizar, filtrar o graficar cualquier objeto que llegue de fuera, hay cuatro preguntas que conviene responder:

1. ¿Son cuentas crudas?
2. ¿Están ya filtradas?
3. ¿Dónde están los metadatos de lote?
4. ¿Qué anotación se usó?

Un objeto que no puede contestarlas no está listo para analizarse, y el problema no se resuelve corriendo más código: se resuelve preguntando a quien lo produjo. Estas mismas cuatro preguntas se aplican a los datos del proyecto en la Capítulo 6.

4.3.6. Llamado de células

Una corrida de 10x observa cientos de miles de códigos de barras y contiene unos pocos miles de células. El resto son gotas sin célula, que no están limpias: contienen RNA libre del medio.

4.3.6.1. Por qué hay tantos códigos de barras y tan pocas células

La desproporción sorprende la primera vez, y no es un defecto de la corrida. Son tres cosas distintas sumándose, y conviene separarlas.

1. La matriz cruda tiene una columna por código de la lista, no por célula. Antes que nada es una propiedad del archivo. `raw_feature_bc_matrix` reserva una columna para cada código de barras del repertorio de esa química: 737 280 en la v2 y 6 794 880 en la v3. Que haya millones de columnas no dice nada del experimento todavía.

2. Las células se cargan diluidas a propósito. El chip genera decenas de miles de gotas, y la suspensión celular se diluye para que a una gota casi nunca le toquen dos células (Macosko et al., 2015; Zheng et al., 2017). Esa dilución es lo que mantiene baja la tasa de dobles, y su precio es que a la mayoría de las gotas no le toca ninguna.

La aritmética es la de Poisson. Si λ es el número medio de células por gota:

$$P(0) = e^{-\lambda} \quad P(1) = \lambda e^{-\lambda} \quad P(\geq 2) \approx \frac{\lambda^2}{2}$$

Con λ pequeña, los dobles crecen con el **cuadrado** de la concentración mientras que las células recuperadas crecen de forma lineal. Bajar λ a la mitad divide los dobles entre cuatro y el rendimiento solo entre dos. Ahí está el compromiso completo del cargado, y es la misma cuenta que reaparece el 1 de septiembre al estimar la tasa esperada de dobles.

3. Una gota sin célula no está vacía de RNA. Durante la disociación algunas células se rompen y su RNA queda libre en la suspensión. Esa «sopa» se reparte entre **todas** las gotas, con célula o sin ella (Young & Behjati, 2020). Una gota sin célula produce entonces cuentas de verdad —del orden de unas decenas de moléculas—, y por eso aparece en la matriz en lugar de ser una columna de ceros.

Eso es justamente lo que vuelve útiles a las gotas vacías: son una muestra del ambiente, medida con el mismo protocolo y en la misma corrida. `emptyDrops` las usa como grupo de comparación (Lun et al., 2019), y `SoupX` las usa para estimar cuánta contaminación entró en las gotas con célula.

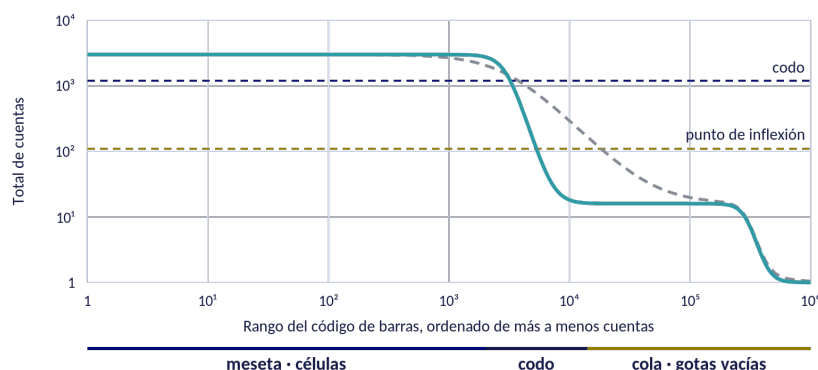
Y un cuarto sumando, más pequeño: los errores de secuenciación en los 16 nucleótidos del código de barras. Con cientos de millones de lecturas aparecen códigos que no existían. El programa los corrige contra la lista blanca, pero cualquier código con al menos una lectura acaba teniendo su columna.

i La conclusión, que es la que importa

Que haya cientos de miles de códigos de barras con cuentas **es lo esperado**. La pregunta no es por qué hay tantos, sino **cuáles de ellos son células**, y eso no lo contesta el archivo: es una decisión estadística que toma quien analiza (Amezquita et al., 2020; Lun et al., 2019).

Ordenando todos los códigos por su total de cuentas y graficándolos en escala logarítmica se obtiene la **curva de rango**, con tres regiones: una meseta a la izquierda —los códigos con muchas cuentas—, un codo donde cae, y una cola larga de gotas con muy pocas cuentas. La forma de la curva dice bastante del experimento antes de aplicar ningún método; un codo difuso es señal de problemas.

La curva de rango de códigos de barras



Tras el codo hay otra meseta

La curva no se desploma: se aplan a la altura del RNA ambiental. Esas gotas tienen pocas cuentas, pero tienen.

— — — codo difuso

Células y ambiente no se separan. No hay altura donde trazar la línea, y eso se ve antes de aplicar ningún método.

Figura 4.3: La curva de rango y sus tres tramos. Punteada, la misma corrida con un codo difuso: cuando células y ambiente no se separan, no hay altura donde trazar la línea.

Lo que conviene mirar no es dónde termina la curva, sino que **después del codo vuelve a aplanarse**. Esa segunda meseta, a la altura del RNA ambiental, son las gotas sin célula: tienen pocas cuentas —del orden de unas decenas— pero las tienen, y son las que van a servir de grupo de comparación.

i Dos lecturas del eje que confunden

El cero no aparece, y no puede aparecer. El eje vertical es logarítmico, y el logaritmo de cero no existe. Una curva de rango nunca «baja a cero»: baja hasta **1**, que es el mínimo real —un código de barras que está en la matriz tiene al menos una cuenta—.

El eje horizontal también es logarítmico. Por eso la meseta de células, que son unos pocos miles de códigos, ocupa la mitad izquierda del gráfico aunque sea una fracción minúscula del total. En escala lineal serían invisibles.

Hay dos maneras de trazar la línea:

	Umbral fijo	emptyDrops
Criterio	un número de cuentas	prueba estadística contra el perfil ambiental
Qué usa	solo el total	el perfil de expresión completo
Células pequeñas	las pierde	las conserva si su perfil difiere del ambiente

La diferencia importa porque una célula pequeña real y una gota vacía pueden tener el mismo total de cuentas. Lo que las distingue no es cuánto expresan, sino **qué** expresan: **emptyDrops** contrasta el perfil de cada código de barras contra el perfil del ambiente y declara célula a los que se apartan de él con una tasa de descubrimientos falsos declarada (Lun et al., 2019).

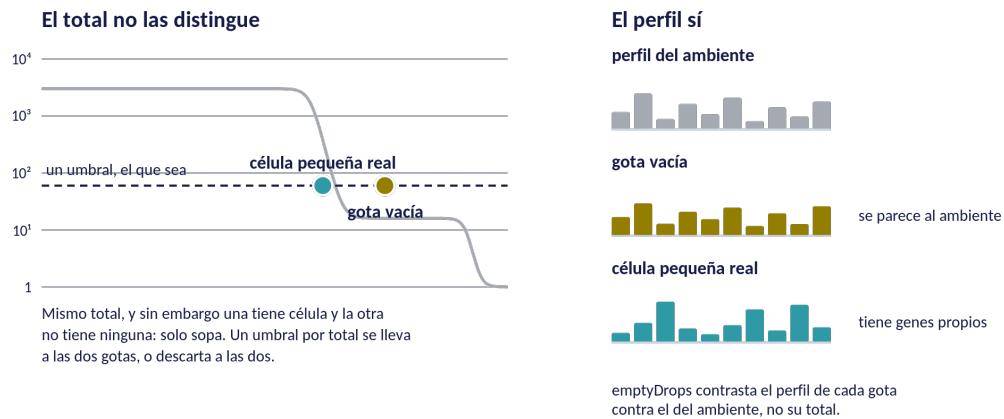


Figura 4.4: A la izquierda, las dos gotas a la misma altura: ningún umbral horizontal las separa. A la derecha, sus perfiles frente al del ambiente — la gota vacía lo repite, la célula pequeña tiene genes propios.

! Importante

Ese contraste necesita las gotas vacías para estimar el perfil ambiental. Con la matriz **filtered** ya no están, y el método se queda sin grupo de comparación. Por eso la decisión del principio de la sesión no era retórica.

4.3.7. RNA ambiental

Las gotas que se acaban de descartar son también el material del que se lee la contaminación ambiental. El RNA libre del medio no solo está en las gotas vacías: está dentro de las gotas con célula, sumándose a su expresión real.

Es el único paso de control de calidad que **no deja pasar ruido sino que genera señal falsa**: un marcador puede aparecer en un tipo celular que no lo expresa, y la conclusión resultante es limpia y equivocada.

SoupX puede estimar esa fracción sin necesitar etiquetas de agrupamiento, si se le proporcionan conjuntos de genes que no deberían expresarse en la mayoría de las células —hemoglobina fuera de eritrocitos, inmunoglobulina fuera de linfocitos B— (Young & Behjati, 2020). Ese modo manual tiene además una ventaja didáctica sobre el automático: la biología la pone quien analiza.

i La conclusión de la sesión

Qué es una célula no lo dice el archivo: lo decide quien analiza, con un criterio que puede nombrar y defender. Lo mismo vale para qué son cuentas, qué es un lote y qué anotación se usó. Un objeto es utilizable cuando puede contestar esas preguntas, no cuando se abre sin error.

4.4. Errores comunes

Error	Cómo se detecta	Cómo se resuelve
Usar la matriz filtered cuando se quería hacer control de calidad	El control de calidad no encuentra nada porque ya se hizo; las gotas vacías no están	Para emptyDrops o para estimar RNA ambiental se necesita la matriz raw. La carpeta filtered ya tiene aplicado un algoritmo de llamado de células.
Tomar adata.X de CELLxGENE como cuentas crudas	Valores decimales tratados como conteos; toda la estadística posterior es inválida	En CELLxGENE las cuentas crudas suelen estar en raw o en una capa. Regla general: si hay decimales, no son cuentas crudas.
Suponer que una matriz de GEO son cuentas	Igual que el anterior, pero más frecuente todavía	Revisar siempre. Los autores suben TPM, CPM o valores normalizados con la misma frecuencia con que suben cuentas.
Unir muestras de GEO sin prefijar el nombre de la muestra	Códigos de barras repetidos que colisionan; células que se fusionan silenciosamente	Prefijar el identificador de muestra a cada código de barras antes de unir.
Convertir entre objetos sin verificar	Se pierden metadatos, reducciones o capas sin ningún aviso	Después de cada conversión verificar tres cosas: dimensiones, suma total de cuentas, y que los metadatos de lote sobrevivieron.
Abrir un objeto Seurat v4 con Seurat v5 sin actualizarlo	Errores incomprensibles o, peor, comportamiento silenciosamente distinto	UpdateSeuratObject() y después revisar que las capas quedaron donde deben.

4.5. Recursos

Clúster institucional; R con SingleCellExperiment, DropletUtils, scuttle y zellkonverter; Seurat; Python con anndata y scanpy; matrices de PBMC (cruda y filtrada); ejemplos de conjuntos de datos de GEO, de CELLxGENE y de un archivo .rds; guion en Quarto provisto; documento de rutas de importación.

En línea

Modalidad en línea Pantalla compartida para la parte guiada. Cada equipo recibe su conjunto de datos por enlace y trabaja en sala simultánea. Las cuatro preguntas obligatorias se responden en un formulario o en el tablero, para que las respuestas de todos los equipos queden comparables al volver a la sala principal.

4.6. Si algo falla

Si falla	Plan B
El trabajo del martes no terminó	Matrices precalculadas en el directorio compartido, generadas en el ensayo del 26 de agosto. La sesión continua sin cambios.
zellkonverter falla por conflicto con conda	Mostrar la conversión precalculada; el archivo .h5ad de respaldo debe estar generado desde el 24 de agosto.
No hay tiempo para las siete rutas	Se desarrollan tres en vivo (FASTQ, GEO y CELLxGENE) y las demás quedan en la tabla del documento de referencia.

4.7. Referencias

- Llamado de células y **emptyDrops**: Lun et al. (2019).
- RNA ambiental y **SoupX**: Young & Behjati (2020).
- Estructuras de datos y flujo de trabajo en Bioconductor: Amezquita et al. (2020).
- Panorama general de buenas prácticas: Luecken & Theis (2019).

La tabla completa de rutas de importación, con las cuatro que no se desarrollan en clase, está en el [anexo de rutas de importación](#).

5 Control de calidad: del experimento a los datos

Sesión 05 · Martes 1 de septiembre de 2026 · sesión práctica, 120 minutos Imparten: Luis Alberto Meza Cova, José Antonio Ovando Ricárdez y Alfredo Morales Salazar

i Objetivo

Reconocer que la calidad de un experimento de célula única se decide en el laboratorio y no en el análisis; identificar qué decisión experimental produce cada artefacto observable en los datos; y aplicar los criterios bioinformáticos que permiten detectarlos, decidiendo con criterio explícito y defendible qué células se descartan.

5.1. Cómo se desarrolla la sesión

Práctica guiada por tres instructores, con los equipos repartidos en salas simultáneas. La sesión recorre la cadena completa: qué decisión se tomó en el laboratorio, qué artefacto produce en la matriz y con qué herramienta se detecta. Se revisan el tiempo de isquemia y el almacenamiento, el método y la temperatura de disociación y su firma de estrés, la elección entre células íntegras y núcleos aislados, la viabilidad al cargar y la carga del canal. Después, las métricas de calidad por célula (cuentas totales, genes detectados y proporción de lecturas mitocondriales, ribosomales y de hemoglobina) y qué revela cada una sobre lo que ocurrió en el banco de trabajo. Umbrales fijos frente a umbrales adaptativos basados en la desviación absoluta mediana, y por qué el umbral debe calcularse por lote. Discusión del caso en que una proporción mitocondrial elevada corresponde a biología legítima y no a daño celular. Identificación de dobletes y su relación con el número de células cargadas. Actividad central: cada equipo aplica un criterio de filtrado distinto al mismo conjunto de datos; en plenaria se comparan cuántas células descartó cada criterio y, sobre todo, qué poblaciones celulares desaparecieron con cada uno.

- 6 bloques en 120 minutos, con descansos entre ellos
- Se alterna explicación y trabajo propio

5.2. Caso de aplicación

Caso de aplicación: el tipo celular que era el protocolo: en 2017 se documentó que la disociación enzimática a 37 grados induce una respuesta de estrés —FOS, JUN, JUNB, EGR1 y proteínas de choque térmico— en una fracción de las células. Esa fracción aparece después como un grupo propio, con marcadores coherentes y reproducible entre muestras, y se ha publicado más de una vez como un estado celular novedoso. No lo es: es el protocolo. La consecuencia práctica para el curso es doble. Primero, el artefacto no se corrige filtrando, porque las células afectadas están vivas y tienen buenas

métricas. Segundo, se previene en el laboratorio, usando proteasa activa en frío, y se detecta en el análisis puntuando esa firma antes de interpretar cualquier grupo. Es el ejemplo más claro de que la calidad se decide antes de secuenciar.

5.3. Desarrollo

5.3.1. El tipo celular que era el protocolo

En 2017 se documentó algo incómodo: la disociación enzimática a 37 °C induce una respuesta de estrés en una fracción de las células. Los genes son reconocibles —*FOS*, *JUN*, *JUNB*, *EGR1* y proteínas de choque térmico— y la fracción afectada aparece después como un grupo propio, con marcadores coherentes y reproducible entre muestras (Brink et al., 2017).

Se ha publicado más de una vez como un estado celular novedoso. No lo es: es el protocolo.

El caso importa por dos razones prácticas, y conviene enunciarlas por separado.

La primera es que **el artefacto no se corrige filtrando**. Las células afectadas están vivas, tienen buenas cuentas, buenos genes detectados y una proporción mitocondrial normal. Ningún umbral de calidad las separa, porque por todas las métricas de calidad son células buenas. Lo que tienen de anómalo es su programa transcripcional, y eso no es un criterio de filtrado.

La segunda es que **se previene en el laboratorio**. Disociar con proteasa activa en frío reduce la respuesta de forma sustancial (O’Flanagan et al., 2019). Es una decisión que se toma antes de secuenciar y que ningún análisis posterior puede recuperar.

De ahí la afirmación que organiza toda la sesión: la calidad de un experimento de célula única se decide en el laboratorio. El análisis la mide; no la produce.

5.3.2. La cadena, decisión por decisión

Cada decisión experimental deja una huella medible. La tabla siguiente es el mapa de la sesión: se lee de izquierda a derecha, del banco de trabajo a la matriz, y es también el orden en que conviene revisar un conjunto de datos ajeno.

Decisión	Artefacto	Cómo se detecta
Tiempo de isquemia prolongado	muerte celular	proporción mitocondrial elevada
Almacenamiento antes de procesar	pérdida de poblaciones frágiles	composición incoherente con el tejido
Disociación a 37 °C	firma de estrés	puntuación de la firma, que atraviesa poblaciones
Células íntegras en tejido fibroso o congelado	faltan tipos celulares	contraste con la histología
Baja viabilidad al cargar	RNA ambiental abundante	fracción de gotas vacías, perfil del ambiente

Hay una asimetría que conviene notar. Los tres primeros artefactos se ven en las métricas por célula; los tres últimos, no del todo. El sesgo de composición —faltan poblaciones enteras— es invisible para cualquier métrica por célula, porque las células que quedaron pueden ser perfectas. Solo se detecta comparando contra lo que se sabe del tejido, y esa comparación es conocimiento biológico, no una función de R. El estudio sistemático de estos sesgos está en Denisenko et al. (2020).

5.3.3. Las métricas por célula, leídas como evidencia

Las cuatro métricas habituales no son un trámite: cada una responde a una pregunta distinta sobre lo que ocurrió antes.

- **Cuentas totales** por célula. Muy bajas sugieren una gota con poco material; muy altas, un doblete.
- **Genes detectados.** Se lee junto con las cuentas: muchas cuentas y pocos genes es el perfil de una célula dominada por unos pocos transcritos, que suele ser daño.
- **Proporción mitocondrial.** Elevada indica pérdida de integridad de la membrana, con el citoplasma escapando y el RNA mitocondrial quedándose. Pero no siempre: hay tipos celulares metabólicamente activos —cardiomiocitos, hepatocitos— cuya proporción alta es biología legítima.
- **Proporción ribosomal y de hemoglobina.** La segunda delata contaminación por eritrocitos, frecuente en tejido no perfundido.

La clasificación formal de células de baja calidad, con más métricas y su comportamiento conjunto, está en Ilicic et al. (2016).

5.3.4. Umbrales: de dónde tienen que salir

El error más común del módulo es copiar los umbrales de un tutorial. «Mitocondrial menor al 5 %, más de 200 genes» funciona razonablemente en PBMC y puede ser desastroso en tejido cardíaco, donde elimina justamente las células de interés.

La alternativa es derivar el umbral de la distribución de los propios datos. El criterio habitual usa la **desviación absoluta mediana**: se marca como atípica la célula que se aparta de la mediana más de un número dado de desviaciones, típicamente tres (McCarthy et al., 2017). La mediana y la MAD se eligen, en lugar de la media y la desviación estándar, porque son robustas: los propios valores extremos que se quieren detectar no arrastran el umbral.

! El umbral se calcula por lote

Con varios lotes de profundidad distinta, un umbral global se ancla en el lote dominante y elimina casi entero al de menor profundidad. El cálculo se hace **por lote**.

Esto exige que los metadatos de lote existan en el objeto, que es exactamente lo que se decidió al diseñar el experimento en el Capítulo 2. Un diseño sin lote registrado no permite este cálculo, y a esas alturas ya no hay forma de reconstruirlo.

5.3.5. Dobletes

Una gota que captura dos células produce un perfil que mezcla dos programas transcripcionales. Si son de tipos distintos, el resultado puede parecer un tipo celular intermedio y publicarse como tal.

La tasa esperada no es un misterio: depende del número de células cargadas y la plataforma la documenta. Se usa como **referencia**, no como verdad: un método de detección que encuentra el triple de lo esperado está diciendo algo sobre la carga, y uno que encuentra la mitad puede estar fallando. En el módulo se usa **scDblFinder**, que simula dobletes artificiales a partir de los propios datos y puntúa cada célula por su parecido con ellos (Germain et al., 2021).

5.3.6. El control de calidad no se hace una sola vez

La última idea es de método. El control de calidad se suele tratar como un paso inicial que se despacha antes de lo interesante. Conviene volver a él **después de agrupar**: un grupo con métricas raras —cuentas sistemáticamente bajas, mitocondrial alta, o la firma de disociación— casi siempre es un artefacto que sobrevivió al filtrado, y a esas alturas es fácil confundirlo con un tipo celular.

! La conclusión de la sesión

Filtrar es una decisión, no un trámite. Dos criterios razonables aplicados al mismo conjunto de datos producen resultados distintos, y la diferencia no está en cuántas células sobreviven sino en **cuáles**. Por eso el entregable de hoy es el reporte con la justificación, y no la tabla de resultados.

5.4. Errores comunes

Error	Cómo se detecta	Cómo se resuelve
Interpretar la firma de disociación como biología	Un grupo definido por FOS, JUN, EGR1 y proteínas de choque térmico, presente en varias poblaciones	Puntuar la firma de estrés antes de anotar. Si atraviesa poblaciones, es técnica. Se prevé con disociación en frío; en el análisis se anota y se decide si se excluye o se modela.
Copiar los umbrales de un tutorial	Se pierden poblaciones celulares legítimas sin ningún aviso	Los umbrales se derivan de la distribución de los propios datos. Umbrales adaptativos por desviación absoluta mediana.
Calcular umbrales de forma global cuando hay varios lotes	El lote con menor profundidad se elimina casi entero	Calcular por lote. Requiere que los metadatos de lote existan, que es lo que se decidió el 20 de agosto.

Error	Cómo se detecta	Cómo se resuelve
Interpretar todo porcentaje mitocondrial alto como daño	Se descartan tipos celulares metabólicamente activos	Contrastar contra el tipo celular esperado. Si la población perdida es coherente entre réplicas, probablemente es biología.
Usar células íntegras en tejido fibrótico o congelado	Faltan poblaciones esperadas; las proporciones no cuadran con la histología	Núcleos aislados. El sesgo de composición no se detecta con métricas por célula: solo se ve comparando con lo que se sabe del tejido.
Filtrar dobletes sin considerar la tasa esperada	Se eliminan células reales, o quedan dobletes que aparecen como un tipo celular nuevo	La tasa esperada depende del número de células cargadas y la indica la plataforma. Se usa como referencia, no como verdad.
Hacer el control de calidad una sola vez y no volver a mirar	Los problemas aparecen después, al agrupar, y se atribuyen a biología	El control de calidad se revisita después de agrupar. Un grupo con métricas raras suele ser un artefacto.

5.5. Recursos

Clúster institucional; R con scuttle, scater y scDblFinder; lista de genes de respuesta a disociación; resultado de detección de dobletes en Python precalculado para contrastar; Seurat para la traducción del mismo filtrado; datos de PBMC y un conjunto de datos de tejido disociado en caliente para contraste; plantilla de reporte en Quarto; tablero de Miro; guion común escrito y ensayado por los tres instructores.

En línea

Modalidad en línea Tres instructores, tres salas simultáneas, dos equipos por sala. Cada equipo pega en el tablero de Miro su gráfica de métricas y el número de células que perdió. La plenaria compara los tableros, no los relatos: sin ese soporte visual la comparación entre seis criterios no se sostiene en línea.

5.6. Si algo falla

Si falla	Plan B
Los tres instructores dan criterios inconsistentes	Guion común escrito y ensayado antes del 1 de septiembre. Es plan A, no plan B: sin él la plenaria no se puede comparar.

Si falla	Plan B
Los equipos no terminan y la plenaria no cuadra	Tener los resultados de todos los criterios precalculados y publicarlos en el tablero.
No se consigue un conjunto de datos con firma de disociación marcada	Se usa la figura publicada del artículo de 2017 y se puntúa la firma sobre los datos de PBMC, donde aparece débil. El contraste se pierde, pero el método se enseña igual.

5.7. Referencias

- El artefacto de disociación, descrito por primera vez: Brink et al. (2017).
- Cómo se previene en el laboratorio, con proteasa activa en frío: O’Flanagan et al. (2019).
- Evaluación sistemática de sesgos por disociación y almacenamiento: Denisenko et al. (2020).
- Métricas por célula y umbrales adaptativos: McCarthy et al. (2017).
- Clasificación de células de baja calidad: Ilicic et al. (2016).
- Detección de dobletes: Germain et al. (2021).
- Panorama del flujo completo: Luecken & Theis (2019).

El texto de referencia del módulo, con las rutas de Bioconductor, es Amezquita et al. (2020).

Parte III

Autonomía y cierre

6 Práctica integradora y arranque del proyecto

Sesión 06 · **jueves 3 de septiembre de 2026** · taller, 120 minutos Imparten: Yalbi Itzel Balderas-Martínez, José Antonio Ovando Ricárdez y Alfredo Morales Salazar. Luis Alberto Meza Cova se incorpora a las 14:00.

i Objetivo

Aplicar de forma autónoma, sobre un conjunto de datos no visto en clase, el flujo completo trabajado hasta ahora —importación, caracterización, llamado de células y control de calidad—, y producir un análisis reproducible documentado en un repositorio, dejando iniciado el proyecto que se presenta el 17 de septiembre.

6.1. Cómo se desarrolla la sesión

Taller. Se entrega el conjunto de datos del proyecto, sin guion de pasos. Se trabaja en salas simultáneas, una por equipo, con un instructor asignado a cada una: el mismo instructor acompaña al mismo equipo hasta el 17 de septiembre. El bloque expositivo es corto y trata de reproducibilidad operativa —estructura del repositorio, `.gitignore` que excluye los datos crudos, reporte en Quarto, semillas fijas y registro de la sesión de trabajo—. El resto del tiempo es trabajo propio: cada equipo importa sus datos, contesta las cuatro preguntas obligatorias, ejecuta el control de calidad con un criterio que pueda defender y produce un reporte que corra de principio a fin. La sesión cierra con una verificación cruzada en la que cada equipo ejecuta el repositorio de otro y reporta el resultado como un issue de GitHub.

- 7 bloques en 120 minutos, de los que **65** son trabajo por equipos
- Una sala simultánea por equipo, con un instructor asignado

6.2. Caso de aplicación

Caso de aplicación: el análisis que no se puede repetir. El momento Retar de esta sesión reproduce en pequeño lo que ocurre cuando una revista pide el código de un artículo. Cada equipo intenta ejecutar el repositorio de otro equipo, en el mismo clúster, con los mismos paquetes, con datos disponibles y con los autores presentes en la sesión. Aun así, una parte no corre: rutas absolutas, archivos que quedaron fuera del control de versiones, pasos que se ejecutaron a mano y no quedaron escritos. La lección no se puede transmitir con una diapositiva y por eso ocupa quince minutos de clase.

6.3. Desarrollo

6.3.1. Por qué esta sesión existe

Las cinco sesiones anteriores son guiadas: guion provisto, datos preparados, pasos numerados. Eso es lo correcto al principio, pero tiene un riesgo que conviene nombrar —se puede llegar al final de un módulo entero sin haber decidido nada—. Ejecutar una receta y entender un análisis son cosas distintas, y la diferencia solo se hace visible cuando se retira la receta.

Hoy se retira. Los datos no se han visto antes, no hay guion de pasos y el orden lo decide cada equipo. El equipo docente acompaña, pero no dicta: hace preguntas. Es la transición supervisada entre seguir instrucciones y trabajar solo, y ocurre con instructores presentes precisamente para que el primer tropiezo tenga a quién preguntarle.

Es también la última sesión antes de que entre el módulo del Dr. Danilo Ceschin, el 8 y el 10 de septiembre. El grupo no vuelve con nosotros hasta el 15. Lo que arranque hoy tiene dos semanas de trabajo en paralelo por delante; lo que no arranque hoy tiene menos de dos días.

6.3.2. Qué se versiona y qué no

La regla es corta: **el repositorio guarda el guion que lee el dato, nunca el dato**. Un objeto de células individuales pesa cientos de megabytes, GitHub rechaza archivos de más de 100 MB, y los datos del proyecto son de uso interno.

Categoría	Va al repositorio	Se queda fuera
Código	el reporte, las funciones auxiliares	—
Datos	—	matrices, <code>.h5ad</code> , <code>.rds</code> , <code>.mtx</code> , <code>outs/</code>
Resultados	tablas pequeñas	figuras, que las regenera el render
Documentación	<code>README</code> , tabla de decisiones	—

La plantilla del curso —en `tareas/plantillas/repositorio-proyecto/`— ya trae el `.gitignore` escrito, con un comentario por bloque explicando qué excluye y por qué. Se clona y se edita; está pensada para que la modifiquen, no para que la borren.

Que los datos no estén en el repositorio obliga a declarar de dónde salen. Esa declaración va en el `README`, en una tabla con tres columnas: qué dato, dónde vive en el clúster y cómo se obtuvo. Sin ella, el repositorio es reproducible solo para quien ya sabe dónde están los archivos, que es justamente la persona que no lo necesita.

6.3.3. La ruta absoluta es el primer fallo

De todos los errores de reproducibilidad, la ruta absoluta es el más frecuente y el más fácil de corregir:


```
# Solo corre en la computadora de quien lo escribió
sce <- readRDS("/Users/ana/Documents/proyecto/datos/objeto.rds")

# Corre en cualquier copia del repositorio
RUTA_DATOS <- "datos/objeto.rds"
sce <- readRDS(RUTA_DATOS)
```

La convención del curso va un paso más allá: **todas las rutas se declaran en un solo bloque**, al principio del reporte, y en ningún otro lugar. Así, quien reproduzca el análisis con su propia copia de los datos cambia ese bloque y nada más. La plantilla ya trae ese bloque, etiquetado **rutas**.

Es lo primero que rompe la verificación cruzada del final de la sesión, y por eso conviene resolverlo antes de escribir la primera figura.

6.3.4. Semillas y registro de sesión

Varios de los métodos del módulo tienen componente aleatorio: **emptyDrops** para el llamado de células, **scDblFinder** y **scrublet** para dobletes, y las reducciones no lineales que se ven la semana siguiente. Sin semilla fijada, dos ejecuciones del mismo código sobre los mismos datos producen dos figuras distintas, y ninguna de las dos es la del reporte.

```
set.seed(20260903)
```

El registro de sesión es la otra mitad del problema. **sessionInfo()**, al final del reporte, deja escrito con qué versiones de qué paquetes corrió el análisis. Cuesta una línea y es lo que permite que dentro de un año alguien entienda por qué el mismo código da otro resultado.

Ninguna de las dos cosas se añade al final. La reproducibilidad no es un paso del flujo: es una propiedad de cómo se trabajó.

6.3.5. Las cuatro preguntas

Antes de la primera figura, cada equipo contesta por escrito, en el reporte, las cuatro preguntas obligatorias del Capítulo 4:

1. **¿Son cuentas crudas?** Si hay decimales, no lo son.
2. **¿Ya están filtradas las células?** De esto depende qué pasos de control de calidad se pueden hacer todavía.
3. **¿Dónde están los metadatos** de lote, donante y condición?
4. **¿Qué versión de la anotación de genes** se usó?

No son preguntas de este curso: son las mismas cuatro que hay que contestar ante cualquier conjunto de datos que llegue en la vida profesional. Contestarlas por escrito, y no de memoria, es lo que evita descubrir en la última semana que la matriz que se venía analizando ya venía filtrada.

Después viene el control de calidad completo, con el criterio que cada equipo decida y **pueda defender**. El criterio de filtrado que se aplicó en la sesión anterior (Capítulo 5) es la primera decisión que van a tener que justificar; hoy se escribe la justificación.

6.3.6. Para quién se escribe un análisis reproducible

Un análisis reproducible tiene tres lectores, y ninguno de los tres está en la sala cuando se escribe:

- **La persona que revisa el artículo**, que va a pedir el código.
- **Quien herede el proyecto**, cuando quien lo escribió ya no esté en el laboratorio.
- **El propio yo de dentro de seis meses**, que es el colaborador más frecuente y el menos comunicativo de los tres.

Ninguno puede preguntar nada. El repositorio tiene que contestar solo. Las reglas de Sandve et al. (2013) y las prácticas de Wilson et al. (2017) son la formulación canónica de esta idea, y las dos son cortas: vale la pena leerlas enteras.

6.3.7. La tabla de decisiones

Cada decisión de análisis —un umbral, un método, un filtro— se anota en la bitácora del **README** en el momento en que se toma, con cuatro columnas: cuándo, qué se decidió, por qué, y qué se probó antes.

Es el criterio A3 de la rúbrica y vale el 15 % de la nota del equipo, pero la razón para llenarla sobre la marcha no es la calificación. La columna que más pesa es la tercera, y reconstruirla de memoria dos semanas después produce justificaciones vacías que se distinguen a simple vista de las escritas en su momento.

6.3.8. Verificación cruzada

En los últimos quince minutos, cada equipo clona y ejecuta el repositorio de otro equipo, y abre un **issue de GitHub** con el resultado: si corrió, y qué falló si no.

Las condiciones son inmejorables: el mismo clúster, los mismos paquetes, los datos disponibles y los autores conectados. Aun así, una parte no corre. Los motivos son siempre los mismos tres —rutas absolutas, archivos que quedaron fuera del control de versiones, pasos que se ejecutaron a mano y no quedaron escritos—, y descubrirlos aquí es barato.

El issue se escribe en el repositorio ajeno, no en el propio, y conviene que lleve tres cosas: si corrió o no, el mensaje de error completo pegado tal cual —no descrito— y qué se intentó antes de rendirse. Ese rastro en el repositorio de otro equipo es exactamente la práctica real.

Si no corre, no es reproducible. La lección es inmediata y no admite discusión, que es la razón por la que ocupa quince minutos de clase en lugar de una diapositiva.

6.4. La conclusión de la sesión

! Importante

Un análisis que no se puede volver a ejecutar no es un resultado: es una anécdota. Lo que se evalúa en el proyecto no es el resultado, sino la justificación de cada decisión que llevó a él.

6.5. Requisito de salida

Ningún equipo se desconecta sin las tres cosas:

1. El **enlace de su repositorio** pegado en el tablero del módulo.
2. El **primer resultado generado**.
3. La **verificación de otro equipo**, reportada en el tablero.

Si un equipo no llega, se queda en una sala con un instructor al terminar la sesión. El arranque no se pospone: después de hoy no hay clase hasta el 15 de septiembre.

6.6. Calendario del proyecto

Fecha	Hito
3 de septiembre	Se entrega el conjunto de datos y arranca el proyecto
3 al 16 de septiembre	Trabajo por equipos, en paralelo al módulo del Dr. Ceschin
8 y 10 de septiembre	Normalización, reducción de dimensionalidad, agrupamiento y anotación
15 de septiembre	Efecto de lote (sesión 07)
17 de septiembre	Presentaciones: 20 minutos y 5 de preguntas por equipo (sesión 08)

6.7. Errores comunes

Error	Cómo se detecta	Cómo se resuelve
Rutas absolutas en el código	El guion solo corre en la computadora de quien lo escribió	Rutas relativas a la raíz del proyecto, declaradas en un solo bloque. Es lo primero que rompe la verificación cruzada.

Error	Cómo se detecta	Cómo se resuelve
Versionar datos crudos en el repositorio	El repositorio se vuelve inmanejable y a veces se filtran datos que no debían publicarse	El <code>.gitignore</code> excluye los datos; el repositorio guarda código, documentación y resultados pequeños.
No fijar semillas	Los resultados cambian entre ejecuciones y no se puede reproducir una figura	Fijar semilla en todo paso con componente aleatorio, y registrarla en el reporte.
Ejecutar pasos a mano sin dejarlos escritos	El análisis existe pero no se puede repetir; nadie recuerda el orden	Todo paso que afecte al resultado va en el guion. Si se hizo a mano, se escribe después.
No registrar las versiones de los paquetes	Un año después el mismo código da otro resultado	Registrar la sesión de trabajo al final del reporte. Es una línea de código y evita discusiones enteras.
Llenar la tabla de decisiones al final	Las justificaciones quedan genéricas y no corresponden a lo que se hizo	Anotar cada decisión en el momento en que se toma, con qué se probó antes.

6.8. Recursos

Clúster institucional; conjunto de datos del proyecto, distinto al trabajado en clase; plantilla de repositorio y de reporte en Quarto; Git y GitHub; rúbrica de evaluación; los paquetes de las sesiones anteriores. No se introduce ninguno nuevo: la sesión es de integración, no de contenido.

En línea

Modalidad en línea. Taller con una sala simultánea por equipo y un instructor asignado, el mismo hasta el 17 de septiembre. El requisito de salida se verifica pegando el enlace del repositorio en el tablero. La verificación cruzada se organiza ahí: cada equipo toma el enlace del equipo siguiente y reporta en el tablero si corrió o no.

6.9. Si algo falla

Si falla	Plan B
Los equipos no alcanzan a crear el repositorio	El requisito de salida se verifica equipo por equipo. Si alguno no llega, se queda en una sala con un instructor al terminar la sesión.
El conjunto de datos del proyecto da problemas inesperados	Debe estar analizado de antemano por la responsable, desde el 31 de agosto. Ese análisis de referencia es también la clave de corrección del 17 de septiembre.

6.10. Referencias

- Diez reglas para investigación computacional reproducible: Sandve et al. (2013).
- Prácticas suficientemente buenas en cómputo científico: Wilson et al. (2017).
- El flujo completo y sus decisiones, como referencia de consulta durante el proyecto: Amezquita et al. (2020).

Parte IV

Anexos

Anexo: entornos de trabajo

Todo lo que necesitas para entrar al servidor del curso y correr las prácticas. Sigue los pasos en orden. Cada uno dice **qué debe salir**: si no sale eso, no avances — ve a [Si algo falla](#) al final.

El módulo se trabaja en el **cluster ken**, del Laboratorio Nacional de Visualización Científica Avanzada (LAVIS, UNAM). Ahí están instalados R, Python, Cell Ranger y los datos, en una sola copia para el grupo. **No hace falta que instales nada en tu computadora**. Si aun así quieres montar el entorno en tu máquina, está al final, en [Si prefieres tu propia computadora](#).

Antes de empezar

Necesitas tres cosas:

1. Tu **usuario** en el cluster, que te llegó por correo.
2. Una **terminal**: en Mac, la app Terminal; en Windows, PowerShell o WSL; en Linux, la que uses.
3. Estar en la red de la UNAM, o con la VPN conectada.

Paso 1 · Entrar al servidor

Sustituye `TUUSUARIO` por el tuyo:

```
ssh TUUSUARIO@ken.lavis.unam.mx
```

Te pedirá la contraseña. **No verás los caracteres mientras escribes** — es normal, no está trabado. Escribe y pulsa Enter.

Qué debe salir: un dibujo con la palabra `ken` y un prompt que termina en `[TUUSUARIO@ken ~]$`.

Paso 2 · Cargar el entorno del curso

Una sola línea, cada vez que entres:

```
source /mnt/data/bioinfo3/compartido/setup-curso.sh
```

Eso carga R, Python y Cell Ranger, y define dos atajos que usarás en las prácticas: `$REFERENCIA` y `$DATOS`.

Qué debe salir: nada. En Unix, el silencio es señal de que salió bien.

i Por qué `source` y no `bash`

`source` ejecuta el guion **dentro de tu sesión**, así que lo que define se queda contigo. `bash setup-curso.sh` abriría una sesión aparte que muere al terminar, y te dejaría igual que antes. Por lo mismo hay que repetirlo en cada terminal nueva: las variables viven en la sesión, no en el disco.

Paso 3 · Comprobar que todo está

```
cellranger --version
R --version | head -1
echo $REFERENCIA
echo $DATOS
```

Qué debe salir:

```
cellranger 10.1.0
R version 4.6.0 (2026-04-24) -- "Because it was There"
/mnt/data/bioinfo3/compartido/10x/refdata-gex-GRCh38-2024-A
/mnt/data/bioinfo3/compartido/10x/datasets/manual_5k_pbmc_NGSC3_ch5_fastqs
```

Si alguna línea sale vacía o dice `command not found`, repite el Paso 2. Es el error más común: se olvida el `source` al abrir una terminal nueva.

Paso 4 · Tu espacio de trabajo

Tienes una carpeta propia. Trabaja **siempre** ahí:

```
cd /mnt/data/bioinfo3/TUUSUARIO
pwd
```

Qué debe salir: `/mnt/data/bioinfo3/TUUSUARIO`

Dos reglas sobre dónde poner las cosas:

- **Tu carpeta personal** es donde escribes todo lo que generes.
- **La carpeta compartida** (`/mnt/data/bioinfo3/compartido`) es de **solo lectura**. Ahí viven los datos y los programas. No intentes escribir ahí ni copiar los datos a tu carpeta: son 35 GB.

⚠ No trabajes en tu `$HOME`

El directorio personal tiene una cuota de **5 GB** y la salida de Cell Ranger no cabe. El trabajo muere a media corrida con `Disk quota exceeded`, que es la peor forma de enterarse. Todo va

a /mnt/data/bioinfo3/TUUSUARIO, que no tiene cuota.

Paso 5 · La regla más importante

Nunca corras un análisis pesado en el nodo de acceso.

Cuando entras por SSH llegas al **nodo de acceso**, que comparten todas las personas conectadas. Está para escribir, mirar archivos y mandar trabajos — no para calcular. Un análisis pesado ahí deja el cluster inservible para el grupo entero.

Todo lo que calcula se manda a la **cola** con **sbatch**. El gestor busca un nodo de cómputo libre, corre tu trabajo ahí y te guarda el resultado.

Hay una razón práctica además de la convivencia: el nodo de acceso tiene **15 GB de memoria** y los de cómputo más de **200 GB**.

! Sin memoria, Cell Ranger no falla: se cuelga en silencio

Su gestor interno espera indefinidamente a que se libere memoria que nadie va a liberar. La terminal se queda muda y parece que trabaja. Se diagnostica así:

```
tail -3 pbmc_5k/_log      # ¿dice «Waiting for jobs to complete»?
uptime                    # carga cerca de cero = espera, no calcula
```

No pierdes lo hecho: al relanzar el mismo comando con el mismo **--id**, Cell Ranger retoma desde la última etapa completa.

Paso 6 · Mandar un trabajo a la cola

Un guion de SLURM tiene dos partes: **arriba** las peticiones al gestor —cuántos núcleos, cuánta memoria, cuánto tiempo— y **abajo** el comando.

```
cd /mnt/data/bioinfo3/TUUSUARIO

cat > mi-trabajo.sh <<'FIN'
#!/bin/bash
#SBATCH --job-name=mi-prueba
#SBATCH --cpus-per-task=2
#SBATCH --mem=4G
#SBATCH --time=00:10:00
#SBATCH --output=mi-prueba-%j.out
#SBATCH --error=mi-prueba-%j.err

source /mnt/data/bioinfo3/compartido/setup-curso.sh
echo "Corriendo en el nodo: $(hostname)"
```

```
cellranger --version
FIN
```

El **source** va **dentro del guion** a propósito: el nodo de cómputo no hereda tu entorno, así que hay que cargarlo otra vez allá.

 Pide la memoria explícitamente

Sin `--mem`, SLURM te da 2 GB por núcleo. Con 8 núcleos son 16 GB, y `cellranger count` no cabe ahí — y ya sabes cómo se manifiesta eso.

Envíalo y mira la cola:

```
sbatch mi-trabajo.sh
squeue -u $USER
```

Qué debe salir: Submitted batch job 12345 y luego una línea con tu trabajo. Ese número es su identificador.

Cuando `squeue` salga vacío, terminó. Lee el resultado:

```
cat mi-prueba-*.out
```

Debe decir el nombre de un nodo de cómputo (`node01`, `node02`...) — **no** `ken`. Si dice `ken`, lo corriste en el nodo de acceso, que es justo lo que no hay que hacer.

Comandos que vas a usar todo el semestre

Comando	Qué hace
<code>squeue -u \$USER</code>	ver tus trabajos en la cola
<code>scancel 12345</code>	cancelar el trabajo con ese identificador
<code>sinfo -s</code>	ver qué nodos hay y cuáles están libres
<code>cat archivo.out</code>	leer la salida de un trabajo terminado
<code>quota -s</code>	ver cuánto espacio te queda

Si algo falla

Lo que ves	Qué significa	Qué hacer
Could not resolve hostname	no hay red o falta la VPN	conéctate a la VPN de la UNAM
Permission denied (publickey,password)	usuario o contraseña mal	revisa el correo con tus datos; si insiste, avisa

Lo que ves	Qué significa	Qué hacer
<code>cellranger: command not found</code>	falta cargar el entorno	corre el Paso 2 otra vez
<code>Disk quota exceeded</code>	tu \$HOME está lleno	trabaja en /mnt/data/bioinfo3/TUUSUARIO muévete a tu carpeta personal
<code>Permission denied</code> al escribir	intentas escribir en compartido/	
<code>sbatch: error: Unable to open file</code>	el guion no existe o el nombre está mal	<code>ls</code> para ver el nombre real
El trabajo termina en segundos y el <code>.err</code> tiene texto	falló	lee el <code>.err</code> : ahí está el motivo
Media hora sin escribir nada	lo corres en el nodo de acceso y se quedó sin memoria	detenlo y mándalo con <code>sbatch</code>

Cuando pidas ayuda

Manda estas tres cosas, o no se puede diagnosticar nada:

1. **El comando exacto** que corriste. Cópialo, no lo describas.
2. **El mensaje de error completo**, tal cual salió.
3. La salida de `pwd` y de `whoami`.

«No me funciona» no alcanza. «Corrí *esto*, esperaba *esto* y salió *esto otro*» se resuelve en un minuto.

Si prefieres tu propia computadora

No hace falta para el curso. Todo está puesto en **ken** y las prácticas están dimensionadas para ahí. Esta sección es para quien quiera además tenerlo en su máquina, por gusto o para seguir trabajando sin conexión al cluster.

Qué se puede montar y qué no

Pieza	¿En tu computadora?
R + Bioconductor	sí, en Windows, macOS y Linux
Python + scanpy	sí, en los tres
Cell Ranger	no , salvo en Linux x86_64

Cell Ranger solo existe como binario de **Linux x86_64**. No hay versión de macOS ni de Windows, así que la cuantificación de las sesiones 03 y 04 se hace en el cluster de todas formas. Lo demás —control de calidad, filtrado, integración— sí corre en cualquier lado.

 Reserva un par de horas

No es difícil, pero es lento: solo **Seurat** tarda entre veinte minutos y una hora en compilarse. No lo dejes para la noche anterior a una práctica.

Paso 1 · Instalar R

Descarga **R 4.6.0** de <https://cran.r-project.org> y, si quieres un entorno gráfico, **RStudio Desktop** de <https://posit.co/download/rstudio-desktop/>.

La versión importa: el curso está congelado contra **R 4.6.0** y **Bioconductor 3.23**. Con otra versión de R, Bioconductor te dará otro *release* y algunas funciones cambian de nombre o de argumentos sin avisar.

Qué debe salir, en la consola de R:

```
R.version.string
#> [1] "R version 4.6.0 (2026-04-24)"
```

Paso 2 · Los paquetes de R

Cuatro bloques, en este orden. Corre uno, espera a que termine, y sigue con el siguiente.

```
# 1. El instalador de Bioconductor, fijado al release del curso
install.packages("BiocManager")
BiocManager::install(version = "3.23")

# 2. Bioconductor
BiocManager::install(c(
  "SingleCellExperiment", "scuttle", "scater", "scran", "bluster",
  "DropletUtils", "scDblFinder", "batchelor", "BiocSingular", "celda",
  "zellkonverter", "basilisk", "BiocNeighbors", "limma", "ComplexHeatmap",
  "glmGamPoi", "GEOquery", "LoomExperiment", "scRNAseq", "TENxPBMCDData"))

# 3. CRAN
install.packages(c("Seurat", "harmony", "SoupX", "remotes"))

# 4. Los que solo viven en GitHub
remotes::install_github(c(
  "immunogenomics/presto",
  "theislab/kBET",
  "immunogenomics/LISI",
  "heiniglab/scPower"))
```

Qué debe salir. Esta comprobación no debe imprimir ninguna línea que diga **AUSENTE**:

```

ver <- function(p) tryCatch(as.character(packageVersion(p)),
                           error = function(e) "AUSENTE")
for (p in c("SingleCellExperiment", "scater", "scuttle", "scraper",
            "DropletUtils", "scDblFinder", "batchelor", "zellkonverter",
            "basilisk", "Seurat", "harmony", "SoupX", "limma",
            "presto", "kBET", "lisi", "scPower")) {
  cat(sprintf("%-22s%s\n", p, ver(p)))
}

```

i Para las clases de septiembre

El módulo del Dr. Ceschin añade tres paquetes más. `hdf5r` es el crítico —sin él, `Seurat::Read10X_h5` no puede abrir un archivo `.h5`— y compila contra la biblioteca HDF5 del sistema, así que en Linux o macOS puede pedirte instalarla antes:

```

install.packages("hdf5r")
BiocManager::install(c("SingleR", "celldex"))

```

Paso 3 · Instalar Python con conda

Instala **Miniforge** desde <https://github.com/conda-forge/miniforge>, que trae conda y mamba ya configurados con el canal conda-forge. Sirve en los tres sistemas.

Luego guarda este archivo con el nombre `environment.yml`, en la carpeta que quieras:

```

name: bioinfo3-sc
channels:
  - conda-forge
  - bioconda
dependencies:
  - python=3.11
  - pip
  - numpy
  - scipy
  - pandas
  - scanpy
  - anndata
  - h5py
  - leidenalg
  - igraph
  - scikit-learn
  - scikit-image           # lo necesita sc.pp.scrublet; no se instala solo
  - matplotlib
  - seaborn
  - jupyterlab
  - ipykernel

```

```
- pip:
  - scar # RNA ambiental; arrastra PyTorch, tarda
```

Y créalo:

```
mamba env create -f environment.yml
mamba activate bioinfo3-sc
```

Qué debe salir:

```
python -c "import scanpy; print('scanpy', scanpy.__version__)"
python -c "import scar; print('scar OK')"
```

Nota

scar arrastra **PyTorch**, que son un par de gigabytes. Si solo te interesan las prácticas de control de calidad, puedes quitar esas dos últimas líneas del archivo y añadirlo después.

Tres trampas conocidas

Importante

- **presto** no está en **CRAN**, ni **kBET**, ni **LISI**, ni **scPower**. Buscarlos con `install.packages()` no da un error claro, solo «package not available». Van con `remotes::install_github()`, que es el bloque 4 de arriba.
- **Seurat v5** cambió su estructura interna — `layers` en lugar de `slots`—, así que los tutoriales escritos para v4 fallan de formas poco obvias. Si sigues uno de internet, comprueba primero para qué versión está escrito.
- **No instales `scrublet` desde PyPI**. No recibe mantenimiento desde 2021, choca con las versiones actuales de `numpy` y `numba`, y puede tumbar la construcción completa del entorno. La detección de dobles se hace con `sc.pp.scrublet`, que ya viene dentro de `scanpy`.

Si formas parte del equipo docente

Con el repositorio del curso a la mano, los dos entornos están congelados en `envs/` y se restauran sin escribir la lista a mano:

```
renv::restore() # usa envs/renv.lock: 323 paquetes, R 4.6.0, Bioc 3.23
```

```
mamba env create -f envs/environment.yml
```

Anexo: glosario

Términos que aparecen en el módulo. Se agregan conforme se usan.

UMI (*unique molecular identifier*). Código de barras molecular que permite contar moléculas originales en lugar de lecturas, y así descontar la amplificación por PCR.

Gota vacía. Gota que recibió código de barras pero ninguna célula. No está vacía de RNA: contiene RNA ambiental, y por eso sirve para estimar la sopa.

Curva de rango (*barcode rank plot*). Los códigos de barras ordenados por su total de cuentas, de mayor a menor, en escala log-log. Tiene tres tramos: una meseta de células, un **codo** donde cae, y una cola larga de gotas vacías.

Codo (*knee*). El punto donde se dobla la curva de rango. **barcodeRanks** lo devuelve como un valor de cuentas, no como un rango, junto con el punto de inflexión —más permisivo—. Los dos se calculan solo a partir del total de cuentas, y por eso pierden células pequeñas reales. Un codo difuso es señal de que células y ambiente no se separan limpiamente.

RNA ambiental (*ambient RNA*, “sopa”). RNA libre en la suspensión, liberado por células rotas antes de la encapsulación. Se reparte entre todas las gotas, así que los marcadores de la población más abundante aparecen en todos los clústeres.

Doblete. Gota que capturó dos células. *Heterotípico* si son de tipos distintos —detectable—; *homotípico* si son del mismo tipo —no detectable por métodos computacionales.

Pseudorréplica. Tratar cada célula como una unidad experimental independiente. Infla el *n*, produce valores *p* diminutos y no responde la pregunta biológica: la réplica es la muestra, no la célula.

Efecto de lote. Variación técnica sistemática entre corridas, kits o personas que procesaron la muestra. Es un problema cuando está confundido con la variable de interés.

Sobrecorrección. Corregir el lote hasta borrar la diferencia biológica. Ocurre justo cuando el lote coincide con la condición.

MAD (*median absolute deviation*). Medida robusta de dispersión que se usa para fijar umbrales de QC adaptativos en lugar de umbrales fijos.

Anexo: rutas de importación y estructuras de datos

Casi nunca se empieza desde los FASTQ. Se empieza desde lo que haya: una carpeta de Cell Ranger, un archivo que mandó un colaborador, una serie de GEO o una descarga de CELLxGENE. Quien solo conoce una ruta se paraliza la primera vez que le llega otra.

La pregunta *¿de dónde vinieron estos datos?* se contesta **antes** que *¿qué análisis les hago?*, porque la primera determina qué se puede hacer y qué ya viene hecho.

Tabla maestra

Origen	Qué llega	Lector en R	Lector en Python	Objeto	¿Cuentas crudas?
FASTQ y Cell Ranger	carpeta outs/ con <code>raw_...</code> y <code>filtered_...</code>	<code>DropletUtils::read10xCounts()</code>	<code>scanpy.read10x_mtx()</code>	<code>SingleCellExperiment</code> / <code>AnnData</code>	
Matriz MTX suelta	<code>matrix.mtx.gz</code> , <code>barcodes.tsv.gz</code> , <code>features.tsv.gz</code>	<code>read10xCounts()</code>	<code>sc.read_10x_mtx()</code>	tres	depende
Objeto de práctica	<code>.rds</code> o descarga por función	<code>SeuratData::LoadData()</code> , paquete <code>scRNAseq</code>		<code>Seurat</code> / <code>SingleCellExperiment</code>	casi nunca
GEO	lo que el autor haya subido	<code>GEOquery::getGEO()</code>	<code>GEOquery::getGEOSupplFiles()</code> directa	el que toque	impredecible
CZ CELLxGENE	<code>.h5ad</code>	<code>zellkonverter::anndata2seurat()</code>	<code>anndata2seurat()</code>	<code>SingleCellExperiment</code> / <code>AnnData</code>	no en X
Colaborador	<code>.rds</code> serializado	<code>readRDS()</code> + <code>UpdateSeuratObject()</code>	vía conversión	<code>Seurat</code>	casi nunca
Formato loom	<code>.loom</code>	<code>LoomExperiment::importLoom()</code>	<code>scimpy.io.Loom().SCE</code> o <code>AnnData</code> , según el lector		variable

Los tres objetos

Concepto	SingleCellExperiment	Seurat v5	AnnData
Matriz	<code>assay(sce, "counts")</code>	<code>LayerData(obj, "counts")</code>	<code>adata.X,</code> <code>adata.layers</code>
Orientación	genes × células	genes × células	células × genes
Metadatos de célula	<code>colData(sce)</code>	<code>obj@meta.data</code>	<code>adata.obs</code>
Metadatos de gen	<code>rowData(sce)</code>	<code>obj[["RNA"]@meta.data</code>	<code>adata.var</code>
Reducciones	<code>reducedDim(sce, "PCA")</code>	<code>obj[["pca"]]</code>	<code>adata.obsm["X_pca"]</code>
Otra modalidad	<code>altExp(sce, "ADT")</code>	otro ensayo	<code>adata.obsm</code> o <code>MuData</code>

⚠ La trampa que más cuesta

AnnData guarda **células en las filas**; los otros dos guardan genes. Toda conversión transpone. Quien compare dimensiones sin saberlo concluye que perdió datos.

Después de cualquier conversión hay que verificar tres cosas: dimensiones, suma total de cuentas, y que los metadatos de lote sobrevivieron. Las conversiones fallan en silencio con más frecuencia de la que uno espera.

Las trampas, por origen

Cell Ranger: raw frente a filtered

La carpeta **filtered** ya tiene aplicado un algoritmo de llamado de células. Para practicar `emptyDrops` o para estimar RNA ambiental hace falta la carpeta **raw**, porque las gotas vacías son precisamente lo que **filtered** descartó.

Es el error más frecuente de quien empieza: hacer control de calidad sobre datos a los que ya se les hizo control de calidad, y no encontrar nada.

El archivo `web_summary.html` se abre **antes** que la matriz. Ahí sale si la corrida tuvo problemas.

Objetos de práctica: ya vienen limpios

Cargan en un segundo y funcionan, pero **ya están filtrados** y a veces normalizados y con reducciones calculadas. Pedir control de calidad sobre ellos no tiene sentido: las células malas no están.

Para enseñar control de calidad se usa la matriz cruda del *dataset* público de 10x, no el objeto curado.

GEO: lo que el autor haya querido subir

1. **La matriz puede no ser cuentas crudas.** Muchos autores suben TPM, CPM o valores normalizados. Regla rápida: si hay decimales, no son cuentas crudas.
2. **Códigos de barras que colisionan** al unir muestras. Hay que prefijar el identificador de muestra antes de unir.
3. **Identificadores de gen inconsistentes** entre archivos de la misma serie.
4. **Los metadatos importantes viven en el texto de la serie**, no en un archivo. Hay que leerlos y transcribirlos.
5. A veces **solo hay un objeto procesado**. Los crudos suelen estar en SRA.

CELLxGENE: estandarizado, pero X no son cuentas

El repositorio impone un esquema común con términos de ontología, lo que hace comparables los conjuntos entre estudios. A cambio, `adata.X` suele contener valores **normalizados**; las cuentas crudas viven en `raw` o en una capa. Quien tome `X` como cuentas obtiene resultados sin sentido y sin mensaje de error.

El Census permite pedir rebanadas del corpus completo sin descargar conjuntos enteros, y devolver el resultado como `SingleCellExperiment`, `Seurat` o `AnnData`.

El .rds de un colaborador

Un objeto guardado con `Seurat` v4 no se comporta igual al abrirlo con v5: cambió la estructura interna a capas. Hay que pasarlo por `UpdateSeuratObject()` y revisar que las capas quedaron donde deben.

Casi siempre viene ya filtrado y normalizado, y qué le hicieron rara vez está documentado.

Incompatibilidad entre versiones, en general

`Seurat` es el ejemplo más visible, pero el problema no es de `Seurat`. **Un archivo serializado no guarda solo datos: guarda la estructura que su paquete usaba cuando se creó.** Si esa estructura cambió, el paquete nuevo tiene que traducirla, y no siempre puede hacerlo sin ayuda.

Dónde aparece en este ecosistema:

Dónde	Qué cambió	Cómo se manifiesta
Seurat v4 → v5	el ensayo pasó de ranuras a capas	funciones que parecen correr pero leen la ranura equivocada
Bioconductor	cada release va atado a una versión de R	dependencias rotas sin mensaje claro al mezclar releases
AnnData / .h5ad	el formato lleva versión de esquema por elemento	un archivo escrito con una versión reciente no abre en una vieja

Dónde	Qué cambió	Cómo se manifiesta
Paquetes compilados	cambia la interfaz binaria de una dependencia	hay que reinstalar lo que se compiló contra ella, aunque el código no haya cambiado

Cómo detectarlo, de lo más rápido a lo más lento

1. Preguntarle al objeto con qué se hizo. Muchos lo llevan escrito:

```
obj@version      # Seurat lo guarda dentro del objeto
packageVersion("Seurat") # la versión que tienes tú

adata.uns.get("schema_version") # CELLxGENE lo anota aquí
anndata.__version__
```

2. Leer el **NEWS.md** del paquete. Es donde se documenta qué cambió entre versiones, y va al grano más que la viñeta. Está en la raíz del repositorio en GitHub, y desde R:

```
news(package = "Seurat")
```

3. Buscar el mensaje de error literal en los **issues** de GitHub. Copiar la línea del error —sin los nombres de tus variables— y buscarla en el repositorio del paquete. Si el problema es de versión, casi siempre hay un issue abierto con la respuesta, y a menudo con la solución exacta.

4. Comprobar que la instalación no mezcla releases, en Bioconductor:

```
BiocManager::valid()
```

Y la parte que sí está en tus manos

Cuando mandes un objeto, manda también la versión. Guardar la salida de `sessionInfo()` junto al archivo cuesta un minuto y le ahorra la tarde a quien lo reciba. Es la misma idea que el `renv.lock` del curso: la versión es parte del dato, no un detalle del entorno.

Las cuatro preguntas obligatorias

Antes de analizar cualquier conjunto de datos, venga de donde venga:

1. **¿Son cuentas crudas?** Si hay decimales, no lo son.
2. **¿Ya están filtradas las células?** Miles de códigos de barras, sí. Cientos de miles o millones, no.
3. **¿Dónde están los metadatos de lote, donante y condición?** Si no aparecen, no se va a poder evaluar efecto de lote (sesión 07).
4. **¿Qué versión de la anotación de genes se usó?** Determina si los identificadores se pueden cruzar con otro conjunto de datos.

Anexo: ¿réplica biológica o técnica?

Catálogo de casos para consultar cuando el escenario no es obvio. Extiende la tabla de la sesión 02 con los sistemas que suelen aparecer en las preguntas: gemelos, cultivos bacterianos, líneas celulares, organoides, plantas y muestreo ambiental.

Cada afirmación de este anexo lleva su fuente. Donde la evidencia está en disputa —y hay dos casos— se dice cuál es la disputa en vez de elegir un bando en silencio.

Las definiciones son las de Blainey et al. (2014), las mismas de la clase:

Réplica biológica: unidad independiente que recibe la condición; entre réplicas varía el material biológico. **Réplica técnica:** medición repetida de la *misma* unidad; entre réplicas solo varía el procedimiento.

Naegle et al. (2015) desarrollan el mismo criterio desde la pregunta que le da título a su artículo —*qué significa n*— y advierten explícitamente contra asignar n = número de células.

Antes de buscar tu caso en la lista

La pregunta no se contesta mirando la muestra. Se contesta mirando **a qué población quieres generalizar**. El mismo tubo puede ser réplica biológica para una pregunta y técnica para otra, y por eso las discusiones se atascan: dos personas responden preguntas distintas sin darse cuenta.

Las tres preguntas que lo resuelven:

1. **¿Sobre qué quiero afirmar algo?** Sobre las personas con esta enfermedad, sobre esta cepa bacteriana, sobre esta línea celular. Esa es tu población.
2. **¿Cuál es la unidad más pequeña que se muestreó de forma independiente de esa población?** Ese es tu n , y se llama **unidad experimental** (Festing & Altman, 2002; Lazic et al., 2018).
3. **Si sustituyera una réplica por otra tomada igual, ¿cambiaría el individuo de la población?** Si no cambia, es técnica.

La consecuencia práctica: las réplicas técnicas se promedian o se suman; no suman n . Contarlas como si lo hicieran es **pseudorreplicación** (Hurlbert, 1984). No es un problema exclusivo de la ecología ni del single-cell: Aarts et al. (2014) auditaron revistas de alto perfil en neurociencia y encontraron que los datos anidados —muchas neuronas de un mismo animal— se analizan de rutina con métodos que asumen independencia, lo que infla el error de tipo I. La estructura es idéntica a la de muchas células de un mismo donante.

Tres respuestas posibles, y la tercera es la que más se usa:

- **Sí** — réplica biológica plena.

- **No** — técnica; mide el método, no la biología.
- **Sí, con reserva** — es una unidad independiente, pero comparte algo que limita a qué población puedes generalizar. Casi todos los casos difíciles caen aquí.

Humanos · donantes y muestreo

El caso	¿Réplica biológica?	Por qué
Dos donantes distintos	sí	varía el individuo: genotipo, edad, exposición
Dos alícuotas de la misma extracción de sangre	no	nada biológico varía
La misma sangre, dos carriles de 10x	no	mide el ruido del carril
El mismo donante, sangre en dos días	no, para comparar donantes	sigue siendo un individuo; es una medida repetida
El mismo donante antes y después del tratamiento	no son réplicas	es un diseño pareado: el sujeto es su propio control
Dos biopsias de regiones distintas del mismo tumor	no, al nivel del paciente	capturan heterogeneidad intratumoral (Gerlinger et al., 2012)
Ojo derecho y ojo izquierdo de la misma persona	no, al nivel de la persona	pareadas dentro del individuo
Dos donantes de la misma familia	sí, con reserva	son individuos, pero comparten genética y ambiente
Dos donantes de la misma cohorte hospitalaria	sí	con la reserva de a qué población generaliza esa cohorte
Placenta y sangre de cordón del mismo recién nacido	no	dos tejidos de un individuo: es una comparación, no una réplica

Sobre el tumor. Gerlinger et al. (2012) secuenciaron múltiples regiones del mismo carcinoma renal y encontraron que entre el **63 % y el 69 %** de las mutaciones somáticas no se detectaban en todas las regiones: una biopsia única captura en promedio poco más de la mitad. La región muestreada no es el tumor, y el tumor no es el paciente.

Gemelos

El caso que más discusión genera, y con razón: la respuesta cambia según la pregunta.

El caso	¿Réplica biológica?	Por qué
Dos gemelos monocigóticos , pregunta sobre un tratamiento	sí, con reserva	son dos individuos con historias ambientales propias; el genotipo casi no varía
Dos gemelos monocigóticos, pregunta sobre efecto del genotipo	casi técnica	el factor que te interesa es casi idéntico entre ambos: no hay contraste
Dos gemelos dicigóticos	sí	comparten la mitad del genoma, como cualquier par de hermanos
Diez pares de gemelos monocigóticos discordantes para una enfermedad	sí , y es un diseño potente	el gemelo sano controla el genotipo; el n es el par (Castillo-Fernandez et al., 2014)
El mismo gemelo, muestreado dos veces	no	un individuo medido dos veces

Por qué digo «casi» y no «idéntico». Los gemelos monocigóticos **no son genéticamente idénticos**. Jónsson et al. (2021) secuenciaron genomas completos de pares y sus familias y encontraron que difieren en promedio en unas **5.2 mutaciones** surgidas en el desarrollo temprano, y que alrededor del **15 %** de los pares tiene un número sustancial de mutaciones privativas de uno solo. Bruder et al. (2008) ya lo habían mostrado antes para variantes de número de copia. Es poco frente a los millones de diferencias entre dos personas cualesquiera —por eso «casi técnica»— pero no es cero.

La regla que ordena los cinco casos: un gemelo monocigótico es réplica biológica para todo aquello que **no** esté fijado por el genotipo, y réplica casi técnica para lo que sí. Son individuos, pero tu **n** genético sigue siendo prácticamente uno.

En el diseño de gemelos discordantes, además, la unidad experimental es el **par**: analizarlo como veinte individuos independientes es pseudorreplicación (Castillo-Fernandez et al., 2014).

Animales de laboratorio

El caso	¿Réplica biológica?	Por qué
Dos ratones de la misma camada	sí	son individuos distintos
Dos ratones de camadas distintas	sí	y además capturan la variación entre camadas
Diez ratones de una cepa isogénica	sí, con reserva	son individuos, pero el resultado no generaliza más allá de la cepa (Voelkl et al., 2020)
Ratones de la misma jaula, tratamiento aplicado al agua de la jaula	no, al nivel del ratón	la unidad tratada es la jaula (Festing & Altman, 2002)
Crías de la misma madre tratada durante la gestación	no, al nivel de la cría	la unidad es la camada (Lazic & Essioux, 2013)

El caso	¿Réplica biológica?	Por qué
Ratones de jaulas distintas, tratamiento por inyección individual	sí	cada animal recibe la condición por separado
El mismo ratón, sangre en dos tiempos	no	un individuo medido dos veces
Dos cortes del mismo cerebro	no	dos trozos de un individuo
Dos ratones knockout de la misma línea fundadora	sí, con reserva	individuos, pero con un único evento de edición detrás
Dos peces cebra de la misma puesta	sí, con reserva	individuos, con fuerte efecto de puesta

El error caro. Si el tratamiento va en el agua, en el alimento o en la madre, la unidad experimental es la jaula o la camada, por más animales que tengan dentro. Lazic & Essioux (2013) revisaron estudios de un modelo animal muy usado y encontraron que **solo el 9 %** identificaba correctamente la camada como unidad. No es un tecnicismo: cambia el **n** en un orden de magnitud.

Sobre las cepas isogénicas. Voelkl et al. (2020) muestran que estandarizar el material biológico —una sola cepa, un solo sexo, una sola edad— reduce la variación dentro del experimento pero **empeora** la reproducibilidad entre laboratorios, porque el resultado queda amarrado a esas condiciones. La reserva no es cosmética.

Cultivo celular y líneas

Aquí la palabra «biológica» significa algo distinto que en humanos, y de ahí viene buena parte de la confusión.

El caso	¿Réplica biológica?	Por qué
Dos alícuotas del mismo frasco, secuenciadas por separado	no	técnica pura
El mismo frasco repartido en dos pozos, tratados igual, cosechados el mismo día	no	son <i>réplicas de pozo</i> : comparten cultivo, pase y día
Cultivos iniciados desde viales congelados distintos, en semanas distintas	sí, con reserva	son experimentos independientes de una sola línea celular (Lazic et al., 2018)
Dos pases muy separados de la misma línea	sí, con reserva	el pase introduce deriva genética y transcripcional (Ben-David et al., 2018)
Dos líneas celulares de donantes distintos	sí	ahí sí varía el material biológico entre individuos (Kilpinen et al., 2017)
Dos clones de iPSC del mismo donante	sí, con reserva	independientes como clones, pero un solo genoma detrás

El caso	¿Réplica biológica?	Por qué
Líneas linfoblastoides de donantes distintos	sí, con reserva	capturan variación humana, pero también estado metabólico y carga de EBV (Choy et al., 2008)
La misma línea cultivada en dos laboratorios	no, si la pregunta es biológica	el laboratorio explica buena parte de la varianza (Volpato & Webber, 2020)

La deriva por pase no es un detalle. Ben-David et al. (2018) compararon 27 cepas de «la misma» línea MCF7 frente a 321 compuestos: **al menos el 75 %** de los compuestos que inhibían fuertemente a unas cepas eran **completamente inactivos** en otras. «Línea celular X» no nombra una unidad experimental estable.

Cuánto pesa el laboratorio. Volpato & Webber (2020) compararon neuronas derivadas de **las mismas** líneas de iPSC, con **el mismo** protocolo, en cinco laboratorios: el laboratorio de origen explicó **hasta el 60 %** de la variación transcriptómica, y los dos contribuyentes principales fueron el número de pase y el uso de progenitores congelados.

Cuánto pesa el donante. Kilpinen et al. (2017) caracterizaron 711 líneas de iPSC de 301 donantes sanos: entre el **5 % y el 46 %** de la variación en fenotipos —capacidad de diferenciación, morfología— proviene de diferencias **entre individuos**. Es el argumento cuantitativo de por qué la línea de otro donante sí es réplica biológica y otro clon del mismo donante no lo es del todo.

Y cuántas hacen falta. Brunner et al. (2023) hicieron el análisis de potencia explícito para diseños con iPSC y concluyen que los estudios caso-control publicados están, en general, **sub-potenciados**. Si vas a diseñar uno, ese es el lugar por donde empezar.

La reserva que hay que decir en voz alta: tres cultivos independientes de una línea permiten afirmar algo sobre **esa línea**, no sobre el tejido del que salió ni sobre la especie. El **n** frente a la población de personas sigue siendo **uno** (Lazic et al., 2018; Naegle et al., 2015).

Organoides

El caso	¿Réplica biológica?	Por qué
Dos organoides del mismo cultivo	sí, con reserva	la variación organoide a organoide es alta (Gehling et al., 2022)
Dos lotes de diferenciación de la misma línea	sí, con reserva	pero el lote pesa menos de lo que suele suponerse
Organoides de pases muy distintos	sí, con reserva	el pase domina la variación (Gehling et al., 2022)
Organoides de líneas de donantes distintos	sí	varía el individuo

⚠ Aquí la intuición común está mal

Es habitual afirmar que «el lote de diferenciación domina la variación». La evidencia disponible no lo respalda. Gehling et al. (2022) secuenciaron organoides individuales y encontraron que la variación **entre lotes** fue baja —incluso preparados por personas distintas— y que lo grande fue la variabilidad **de un organoide a otro dentro del mismo cultivo**, con un efecto profundo del **número de pase**: los organoides de pase 4 agrupaban estrechamente y los de pase 11 se dispersaban.

Y la variabilidad tampoco es una ley de la naturaleza, sino una propiedad del protocolo. Quadrato et al. (2017) describieron variabilidad sustancial entre organoides cerebrales por autoorganización estocástica; Velasco et al. (2019), con otro protocolo, encontraron que el **95 %** de sus organoides generaba un compendio de tipos celulares prácticamente indistinguible, con variabilidad comparable a la que hay entre cerebros humanos individuales.

Lo que se puede afirmar sin exagerar: el organoide individual no es una unidad experimental confiable por sí solo, hay que agregar varios, y el número de pase debe reportarse siempre.

Microbiología

El caso	¿Réplica biológica?	Por qué
Dos extracciones de DNA del mismo cultivo de la noche	no	técnica pura
Dos cultivos inoculados el mismo día desde el mismo preinóculo	no	comparten el punto de partida; son ramas del mismo cultivo
Dos cultivos independientes desde el mismo criovial, en días distintos	sí, con reserva	es la convención del campo; captura variación de crecimiento, no genética
Dos colonias distintas de la misma placa, del mismo clon	sí, con reserva	eventos de crecimiento independientes de un genotipo único
Dos aislados de la misma especie de pacientes distintos	sí	varía la cepa y su historia
El mismo cultivo muestreado en fase log y en estacionaria	no son réplicas	es una serie temporal: son condiciones distintas
Dos muestras del mismo consorcio microbiano	depende	si el consorcio es la unidad, son técnicas; si se muestrearon consorcios distintos, biológicas

El punto que ordena toda esta tabla: en microbiología, «réplica biológica» significa por convención **cultivo independiente**, que es un listón más bajo que «individuo independiente» en estudios humanos. No es un abuso del término, pero cambia a qué se puede generalizar: tres cultivos de *E. coli* K-12 hablan de K-12, no de *E. coli*. Es la misma reserva que Naegle et al. (2015) plantean para cualquier sistema con un solo genotipo detrás.

Plantas

El caso	¿Réplica biológica?	Por qué
Dos hojas de la misma planta	no	dos partes de un individuo
Dos plantas de la misma línea endogámica, misma charola	sí, con reserva	individuos casi clonales, con efecto de charola
Dos plantas de la misma línea, cámaras de crecimiento distintas	sí, con reserva	y la cámara puede confundirse con el tratamiento (Leek et al., 2010)
Dos plantas de accesiones distintas	sí	varía el genotipo
Esquejes del mismo individuo	no, genéticamente	son clones: mide variación ambiental, no genética
Semillas del mismo lote sembradas por separado	sí, con reserva	independientes al germinar, emparentadas por origen

La charola y la cámara son el equivalente vegetal de la jaula: si la condición se aplica al contenedor, el contenedor es la unidad (Festing & Altman, 2002).

Muestreo ambiental y de campo

El caso	¿Réplica biológica?	Por qué
Dos submuestras del mismo núcleo de suelo	no	técnica
Dos núcleos de la misma parcela	no, al nivel de parcela	son submuestras de una unidad
Núcleos de parcelas distintas con el mismo tratamiento	sí	la parcela es la unidad experimental
Dos muestras de agua del mismo punto y momento	no	técnica
Muestras del mismo punto en meses distintos	no son réplicas	es una serie temporal

Este es el terreno donde Hurlbert (1984) acuñó el término, y el ejemplo canónico sigue siendo el mejor: medir diez veces dentro de un mismo estanque no da $n = 10$ estanques.

La capa técnica de la secuenciación

Todo lo de esta tabla es técnico. Se incluye porque a veces se presenta como si sumara n .

El caso	¿Réplica biológica?
La misma biblioteca en dos carriles	no
El mismo RNA, dos preparaciones de biblioteca	no
La misma muestra con dos kits de extracción	no, y además introduce una confusión
Resequenciar para ganar profundidad	no: da más lecturas, no más n
El mismo pozo GEM, dos corridas de secuenciación	no
Dos pozos GEM cargados de la misma suspensión	no, pero miden el ruido entre pozos
Dos muestras de donantes distintos multiplexadas en un carril	sí : la réplica son los donantes

Por qué la réplica técnica se usa poco en secuenciación. Marioni et al. (2008) evaluaron la reproducibilidad técnica de RNA-seq y concluyeron que la variación técnica es lo bastante pequeña como para que, para muchos propósitos, **baste secuenciar cada muestra una sola vez**. El presupuesto rinde más en donantes que en repeticiones.

Por qué el multiplexado es buena idea. El *hashing* mezcla réplicas biológicas dentro de un mismo lote técnico, que es exactamente lo que conviene: elimina el lote como variable confundida sin tocar el **n** (Kang et al., 2018; Stoeckius et al., 2018). El problema que resuelve está documentado desde Leek et al. (2010) —los efectos de lote hacen daño sobre todo cuando están **correlacionados con la condición**— y para single-cell en particular por Hicks et al. (2018) y Tung et al. (2017).

Específico de single-cell

El caso	¿Réplica biológica?	Por qué
Diez mil células de un donante	no: n = 1	las células comparten donante y lote; no son independientes
Células de dos clústeres del mismo donante	no son réplicas	son estados distintos dentro de un individuo
Un donante secuenciado en dos pozos GEM	no	el donante sigue siendo uno
Cinco donantes, mil células cada uno	sí: n = 5	la unidad es el donante; las células se agregan
Núcleos y células enteras de la misma muestra	no	dos protocolos sobre el mismo material

Es la línea que sostiene la sesión 02: la réplica es el donante, no la célula. De ahí sale el pseudobulk — se agregan las células por donante y se hace la prueba con **n** = número de donantes (Crowell et al., 2020; Squair et al., 2021).

i Lo que está en disputa, y lo que no

No está en disputa que las células de un mismo individuo no son réplicas independientes: en eso coinciden todas las fuentes de esta sección.

Sí está en disputa cuál es el remedio. Squair et al. (2021) y Crowell et al. (2020) argumentan a favor del **pseudobulk**; Zimmerman et al. (2021) proponen **modelos mixtos** con el individuo como efecto aleatorio; Murphy & Skene (2022) vuelven sobre la comparación con una métrica distinta y encuentran que los métodos de pseudobulk rinden mejor, con réplica de los autores originales.

En este módulo se enseña pseudobulk, y conviene decir en clase que es una elección con literatura en contra, no un hecho cerrado.

Los cinco errores que más aparecen

1. **Contar células como n.** El más caro y el más frecuente. Produce valores p diminutos que no significan nada (Squair et al., 2021).
2. **Confundir «independiente» con «distinto».** Dos tubos distintos del mismo individuo son distintos y no son independientes (Aarts et al., 2014).
3. **Olvidar dónde se aplicó el tratamiento.** Si va en el agua de la jaula, en la madre o en el medio del frasco, la unidad es la jaula, la camada o el frasco (Festing & Altman, 2002; Lazic & Essioux, 2013).
4. **Llamar biológica a una réplica que comparte todo el genotipo sin decir la reserva.** No está mal usarlas; está mal no declarar a qué población generalizan (Voelkl et al., 2020).
5. **Creer que la réplica técnica no sirve.** Sí sirve: mide el ruido del método (Blainey et al., 2014). Lo que no puede es sustituir a la biológica.

Cuando tu caso no está en la lista

Responde estas tres por escrito antes de discutirlo con nadie:

1. ¿Sobre qué población quiero afirmar algo?
2. ¿Qué varía entre mis dos muestras, y qué se mantiene igual?
3. Si el efecto que busco fuera cero, ¿qué diferencia esperarías ver entre ellas de todos modos?

Si la respuesta a la segunda es «solo el procedimiento», es técnica. Si es «algo del material biológico», es biológica — y la tercera te dice de qué tamaño es la reserva que debes declarar.

Referencias

- 10x Genomics. (s. f.). *What fraction of mRNA transcripts are captured per cell?* 10x Genomics Knowledge Base. Recuperado <https://kb.10xgenomics.com/hc/en-us/articles/360001539051>
- Aarts, E., Verhage, M., Veenvliet, J. V., Dolan, C. V., & Sluis, S. van der. (2014). A solution to dependency: using multilevel analysis to accommodate nested data. *Nature Neuroscience*, 17(4), 491-496. <https://doi.org/10.1038/nn.3648>
- Amezquita, R. A., Lun, A. T. L., Becht, E., Carey, V. J., Carpp, L. N., Geistlinger, L., Marini, F., Rue-Albrecht, K., Risso, D., Soneson, C., Waldron, L., Pagès, H., Smith, M. L., Huber, W., Morgan, M., Gottardo, R., & Hicks, S. C. (2020). Orchestrating single-cell analysis with Bioconductor. *Nature Methods*, 17(2), 137-145. <https://doi.org/10.1038/s41592-019-0654-x>
- Ben-David, U., Siranosian, B., Ha, G., et al. (2018). Genetic and transcriptional evolution alters cancer cell line drug response. *Nature*, 560(7718), 325-330. <https://doi.org/10.1038/s41586-018-0409-3>
- Blainey, P., Krzywinski, M., & Altman, N. (2014). Replication. *Nature Methods*, 11(9), 879-880. <https://doi.org/10.1038/nmeth.3091>
- Brink, S. C. van den, Sage, F., Vértessy, Á., Spanjaard, B., Peterson-Maduro, J., Baron, C. S., et al. (2017). Single-cell sequencing reveals dissociation-induced gene expression in tissue subpopulations. *Nature Methods*, 14, 935-936. <https://doi.org/10.1038/nmeth.4437>
- Bruder, C. E. G. et al. (2008). Phenotypically concordant and discordant monozygotic twins display different DNA copy-number-variation profiles. *The American Journal of Human Genetics*, 82(3), 763-771. <https://doi.org/10.1016/j.ajhg.2007.12.011>
- Brunner, J. W., Lammertse, H. C. A., Berkel, A. A. van, et al. (2023). Power and optimal study design in iPSC-based brain disease modelling. *Molecular Psychiatry*, 28(4), 1545-1556. <https://doi.org/10.1038/s41380-022-01866-3>
- Cao, J., Packer, J. S., Ramani, V., Cusanovich, D. A., Huynh, C., Daza, R., Qiu, X., Lee, C., Furlan, S. N., Steemers, F. J., Adey, A., Waterston, R. H., Trapnell, C., & Shendure, J. (2017). Comprehensive single-cell transcriptional profiling of a multicellular organism. *Science*, 357(6352), 661-667. <https://doi.org/10.1126/science.aam8940>
- Castillo-Fernandez, J. E., Spector, T. D., & Bell, J. T. (2014). Epigenetics of discordant monozygotic twins: implications for disease. *Genome Medicine*, 6(7), 60. <https://doi.org/10.1186/s13073-014-0060-z>
- Choy, E., Yelensky, R., Bonakdar, S., et al. (2008). Genetic analysis of human traits in vitro: drug response and gene expression in lymphoblastoid cell lines. *PLoS Genetics*, 4(11), e1000287. <https://doi.org/10.1371/journal.pgen.1000287>
- Crowell, H. L., Soneson, C., Germain, P.-L., Calini, D., Collin, L., Raposo, C., Malhotra, D., & Robinson, M. D. (2020). muscat detects subpopulation-specific state transitions from multi-sample multi-condition single-cell transcriptomics data. *Nature Communications*, 11, 6077. <https://doi.org/10.1038/s41467-020-19894-4>
- Denisenko, E., Guo, B. B., Jones, M., Hou, R., Kock, L. de, Lassmann, T., et al. (2020). Systematic assessment of tissue dissociation and storage biases in single-cell and single-nucleus RNA-seq workflows. *Genome Biology*, 21, 130. <https://doi.org/10.1186/s13059-020-02048-6>

- Ding, J., Adiconis, X., Simmons, S. K., Kowalczyk, M. S., Hession, C. C., Marjanovic, N. D., Hughes, T. K., Wadsworth, M. H., Burks, T., Nguyen, L. T., Kwon, J. Y. H., Barak, B., Ge, W., Kedaigle, A. J., Carroll, S., Li, S., Hacohen, N., Rozenblatt-Rosen, O., Shalek, A. K., ... Levin, J. Z. (2020). Systematic comparison of single-cell and single-nucleus RNA-sequencing methods. *Nature Biotechnology*, 38(6), 737-746. <https://doi.org/10.1038/s41587-020-0465-8>
- Dobin, A., Davis, C. A., Schlesinger, F., Drenkow, J., Zaleski, C., Jha, S., Batut, P., Chaisson, M., & Gingeras, T. R. (2013). STAR: ultrafast universal RNA-seq aligner. *Bioinformatics*, 29, 15-21. <https://doi.org/10.1093/bioinformatics/bts635>
- Festing, M. F. W., & Altman, D. G. (2002). Guidelines for the design and statistical analysis of experiments using laboratory animals. *ILAR Journal*, 43(4), 244-258. <https://doi.org/10.1093/il ar.43.4.244>
- Gehling, K., Parekh, S., et al. (2022). RNA-sequencing of single cholangiocyte-derived organoids reveals high organoid-to-organoid variability. *Life Science Alliance*, 5(12), e202101340. <https://doi.org/10.26508/lsa.202101340>
- Gerlinger, M., Rowan, A. J., Horswell, S., et al. (2012). Intratumor heterogeneity and branched evolution revealed by multiregion sequencing. *The New England Journal of Medicine*, 366(10), 883-892. <https://doi.org/10.1056/NEJMoA1113205>
- Germain, P.-L., Lun, A., Macnair, W., & Robinson, M. D. (2021). Doublet identification in single-cell sequencing data using scDblFinder. *F1000Research*, 10, 979. <https://doi.org/10.12688/f1000research.73600.1>
- Hagemann-Jensen, M., Ziegenhain, C., Chen, P., Ramsköld, D., Hendriks, G.-J., Larsson, A. J. M., Faridani, O. R., & Sandberg, R. (2020). Single-cell RNA counting at allele and isoform resolution using Smart-seq3. *Nature Biotechnology*, 38, 708-714. <https://doi.org/10.1038/s41587-020-0497-0>
- He, D., Zakeri, M., Sarkar, H., Sonesson, C., Srivastava, A., & Patro, R. (2022). Alevin-fry unlocks rapid, accurate and memory-frugal quantification of single-cell RNA-seq data. *Nature Methods*, 19, 316-322. <https://doi.org/10.1038/s41592-022-01408-3>
- Hicks, S. C., Townes, F. W., Teng, M., & Irizarry, R. A. (2018). Missing data and technical variability in single-cell RNA-sequencing experiments. *Biostatistics*, 19(4), 562-578. <https://doi.org/10.1093/biostatistics/kxx053>
- Hurlbert, S. H. (1984). Pseudoreplication and the design of ecological field experiments. *Ecological Monographs*, 54(2), 187-211. <https://doi.org/10.2307/1942661>
- Ilicic, T., Kim, J. K., Kolodziejczyk, A. A., Bagger, F. O., McCarthy, D. J., Marioni, J. C., et al. (2016). Classification of low quality cells from single-cell RNA-seq data. *Genome Biology*, 17, 29. <https://doi.org/10.1186/s13059-016-0888-1>
- Jónsson, H., Magnusdottir, E., Eggertsson, H. P., et al. (2021). Differences between germline genomes of monozygotic twins. *Nature Genetics*, 53(1), 27-34. <https://doi.org/10.1038/s41588-020-00755-1>
- Kaminow, B., Yunusov, D., & Dobin, A. (2021). *STARsolo: accurate, fast and versatile mapping/quantification of single-cell and single-nucleus RNA-seq data*. bioRxiv. <https://doi.org/10.1101/2021.05.05.442755>
- Kang, H. M., Subramaniam, M., Targ, S., Nguyen, M., Maliskova, L., et al. (2018). Multiplexed Droplet Single-Cell RNA-Sequencing Using Natural Genetic Variation. *Nature Biotechnology*, 36(1), 89-94. <https://doi.org/10.1038/nbt.4042>
- Kilpinen, H., Goncalves, A., Leha, A., et al. (2017). Common genetic variation drives molecular heterogeneity in human iPSCs. *Nature*, 546(7658), 370-375. <https://doi.org/10.1038/nature22403>
- Lazic, S. E., Clarke-Williams, C. J., & Munafò, M. R. (2018). What exactly is «N» in cell culture and animal experiments? *PLOS Biology*, 16(4), e2005282. <https://doi.org/10.1371/journal.pbio.2005282>
- Lazic, S. E., & Essioux, L. (2013). Improving basic and translational science by accounting for litter-

- to-litter variation in animal models. *BMC Neuroscience*, 14, 37. <https://doi.org/10.1186/1471-2202-14-37>
- Leek, J. T., Scharpf, R. B., Bravo, H. C., et al. (2010). Tackling the widespread and critical impact of batch effects in high-throughput data. *Nature Reviews Genetics*, 11(10), 733-739. <https://doi.org/10.1038/nrg2825>
- Luecken, M. D., & Theis, F. J. (2019). Current best practices in single-cell RNA-seq analysis: a tutorial. *Molecular Systems Biology*, 15(6), e8746. <https://doi.org/10.15252/msb.20188746>
- Lun, A. T. L., Riesenfeld, S., Andrews, T., Dao, T. P., Gomes, T., & Marioni, J. C. (2019). EmptyDrops: distinguishing cells from empty droplets in droplet-based single-cell RNA sequencing data. *Genome Biology*, 20, 63. <https://doi.org/10.1186/s13059-019-1662-y>
- Macosko, E. Z., Basu, A., Satija, R., Nemesh, J., Shekhar, K., et al. (2015). Highly Parallel Genome-wide Expression Profiling of Individual Cells Using Nanoliter Droplets. *Cell*, 161(5), 1202-1214. <https://doi.org/10.1016/j.cell.2015.05.002>
- Marioni, J. C., Mason, C. E., Mane, S. M., Stephens, M., & Gilad, Y. (2008). RNA-seq: an assessment of technical reproducibility and comparison with gene expression arrays. *Genome Research*, 18(9), 1509-1517. <https://doi.org/10.1101/gr.079558.108>
- McCarthy, D. J., Campbell, K. R., Lun, A. T. L., & Wills, Q. F. (2017). Scater: pre-processing, quality control, normalization and visualization of single-cell RNA-seq data in R. *Bioinformatics*, 33, 1179-1186. <https://doi.org/10.1093/bioinformatics/btw777>
- Melsted, P., Boeshaghi, A. S., Liu, L., Gao, F., Lu, L., Min, K. H., Veiga Beltrame, E. da, Hjørleifsson, K. E., Gehring, J., & Pachter, L. (2021). Modular, efficient and constant-memory single-cell RNA-seq preprocessing. *Nature Biotechnology*, 39, 813-818. <https://doi.org/10.1038/s41587-021-00870-2>
- Murphy, A. E., & Skene, N. G. (2022). A balanced measure shows superior performance of pseudobulk methods in single-cell RNA-sequencing analysis. *Nature Communications*, 13, 7851. <https://doi.org/10.1038/s41467-022-35519-4>
- Naegle, K., Gough, N. R., & Yaffe, M. B. (2015). Criteria for biological reproducibility: what does "n" mean? *Science Signaling*, 8(371), fs7. <https://doi.org/10.1126/scisignal.aab1125>
- O'Flanagan, C. H., Campbell, K. R., Zhang, A. W., Kabeer, F., Lim, J. L. P., Biele, J., et al. (2019). Dissociation of solid tumor tissues with cold active protease for single-cell RNA-seq minimizes conserved collagenase-associated stress responses. *Genome Biology*, 20, 210. <https://doi.org/10.1186/s13059-019-1830-0>
- Picelli, S., Faridani, O. R., Björklund, Å. K., Winberg, G., Sagasser, S., & Sandberg, R. (2014). Full-length RNA-seq from single cells using Smart-seq2. *Nature Protocols*, 9(1), 171-181. <https://doi.org/10.1038/nprot.2014.006>
- Quadrato, G., Nguyen, T., Macosko, E. Z., et al. (2017). Cell diversity and network dynamics in photosensitive human brain organoids. *Nature*, 545(7652), 48-53. <https://doi.org/10.1038/nature22047>
- Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10), e1003285. <https://doi.org/10.1371/journal.pcbi.1003285>
- Schmid, K. T., Höllbacher, B., Cruceanu, C., Böttcher, A., Lickert, H., Binder, E. B., Theis, F. J., & Heinig, M. (2021). scPower accelerates and optimizes the design of multi-sample single cell transcriptomic studies. *Nature Communications*, 12, 6625. <https://doi.org/10.1038/s41467-021-26779-7>
- Slyper, M., Porter, C. B. M., Ashenberg, O., Waldman, J., Drokhlyansky, E., Wakiro, I., Smillie, C., Smith-Rosario, G., Wu, J., Dionne, D., Vigneau, S., Jané-Valbuena, J., Tickle, T. L., Napolitano, S., Su, M.-J., Patel, A. G., Karlstrom, A., Gritsch, S., Nomura, M., ... Regev, A. (2020). A single-cell and single-nucleus RNA-Seq toolbox for fresh and frozen human tumors. *Nature*

- Medicine*, 26(5), 792-802. <https://doi.org/10.1038/s41591-020-0844-1>
- Squair, J. W., Gautier, M., Kathe, C., Anderson, M. A., James, N. D., Hutson, T. H., Hudelle, R., Kaiser, T., Matson, K. J. E., Barraud, Q., Levine, A. J., La Manno, G., Skinnider, M. A., & Courtine, G. (2021). Confronting false discoveries in single-cell differential expression. *Nature Communications*, 12, 5692. <https://doi.org/10.1038/s41467-021-25960-2>
- Stoeckius, M., Zheng, S., Houck-Loomis, B., Hao, S., Yeung, B. Z., et al. (2018). Cell Hashing with Barcoded Antibodies Enables Multiplexing and Doublet Detection for Single Cell Genomics. *Genome Biology*, 19(1), 224. <https://doi.org/10.1186/s13059-018-1603-1>
- Svensson, V. (2020). Droplet scRNA-seq is not zero-inflated. *Nature Biotechnology*, 38(2), 147-150. <https://doi.org/10.1038/s41587-019-0379-5>
- Svensson, V., Natarajan, K. N., Ly, L.-H., Miragaia, R. J., Labalette, C., Macaulay, I. C., Cvejic, A., & Teichmann, S. A. (2017). Power analysis of single-cell RNA-sequencing experiments. *Nature Methods*, 14(4), 381-387. <https://doi.org/10.1038/nmeth.4220>
- Tung, P.-Y., Blischak, J. D., Hsiao, C. J., Knowles, D. A., Burnett, J. E., Pritchard, J. K., & Gilad, Y. (2017). Batch effects and the effective design of single-cell gene expression studies. *Scientific Reports*, 7, 39921. <https://doi.org/10.1038/srep39921>
- Velasco, S., Kedaigle, A. J., Simmons, S. K., et al. (2019). Individual brain organoids reproducibly form cell diversity of the human cerebral cortex. *Nature*, 570(7762), 523-527. <https://doi.org/10.1038/s41586-019-1289-x>
- Voelkl, B., Altman, N. S., Forsman, A., et al. (2020). Reproducibility of animal research in light of biological variation. *Nature Reviews Neuroscience*, 21(7), 384-393. <https://doi.org/10.1038/s41583-020-0313-3>
- Volpato, V., & Webber, C. (2020). Addressing variability in iPSC-derived models of human disease: guidelines to promote reproducibility. *Disease Models & Mechanisms*, 13(1), dmm042317. <https://doi.org/10.1242/dmm.042317>
- Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLoS Computational Biology*, 13(6), e1005510. <https://doi.org/10.1371/journal.pcbi.1005510>
- Young, M. D., & Behjati, S. (2020). SoupX removes ambient RNA contamination from droplet-based single-cell RNA sequencing data. *GigaScience*, 9(12), giaa151. <https://doi.org/10.1093/gigascience/giaa151>
- Zheng, G. X. Y., Terry, J. M., Belgrader, P., Ryvkin, P., Bent, Z. W., Wilson, R., Ziraldo, S. B., Wheeler, T. D., McDermott, G. P., Zhu, J., Gregory, M. T., Shuga, J., Montesclaros, L., Underwood, J. G., Masquelier, D. A., Nishimura, S. Y., Schnall-Levin, M., Wyatt, P. W., Hindson, C. M., ... Bielas, J. H. (2017). Massively parallel digital transcriptional profiling of single cells. *Nature Communications*, 8, 14049. <https://doi.org/10.1038/ncomms14049>
- Zimmerman, K. D., Espeland, M. A., & Langefeld, C. D. (2021). A practical solution to pseudoreplication bias in single-cell studies. *Nature Communications*, 12, 738. <https://doi.org/10.1038/s41467-021-21038-1>